Glossary

From first principles to expert practice

Built from the manuscript sources · 0 chapter(s) in this edition

Appendix A — Glossary

A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Z

Appendix A — Glossary

> Generated by `build/build.py` from the definition callouts in every chapter; the first (defining) occurrence of each term is used. Canonical usage rules are in `docs/project/terminology.md`.

A

Access-control model — A rule system that answers "may subject *S* perform action *A* on resource *R* (in context *C*)?" The three models in common use differ in what the rules are written in terms of: **roles** (RBAC), **attributes** (ABAC) or **relationships** (ReBAC). *(§8.12.1)*

ACID — The four guarantees of a database transaction, named by Härder and Reuter [Härder and Reuter 1983]: **Atomicity** — all of a transaction's writes take effect, or none do. **Consistency** — a transaction takes the database from one state satisfying all declared invariants (constraints) to another. **Isolation** — concurrently executing transactions do not see each other's partial effects; at the strongest level, the outcome equals some serial execution. **Durability** — once a commit is acknowledged, its effects survive crashes. *(§11.12)*

Active–passive and active–active — In an **active–passive** pair, one balancer carries all traffic while a standby monitors it and takes over the VIP on failure. In **active–active**, all balancers carry traffic at once (spread by DNS, ECMP or anycast), and survivors absorb a failed member's share. *(§19.7)*

Active recall — Deliberately retrieving information from memory without looking at the source, for example by answering a question, drawing a diagram or explaining aloud. Its measured benefit for later retention is called the **testing effect**. *(§3.4.2)*

Address space — The range of memory addresses a process can use. On Linux x86-64 with 4-level page tables, user space spans 2^{47} bytes = 128 TiB of *virtual* addresses; the kernel occupies the upper half and is not accessible from user mode. *(§6.3.2)*

Analysis — The process that converts raw text into the terms stored in (or looked up from) an inverted index. An **analyzer** is a pipeline of zero or more **character filters** (rewrite the raw character stream), exactly one **tokenizer** (split the stream into tokens with positions and offsets) and zero or more **token filters** (modify, remove or add tokens). *(§16.4.1)*

Anycast — An addressing scheme in which the same IP prefix is announced via BGP from many locations at once. Each client's packets are routed to whichever announcing location is "closest" according to BGP path selection. *(§7.5.3)*

Apache Flink — An open-source distributed engine that executes dataflow programs as a graph of long-running, pipelined operators, treating batch as a bounded special case of streaming. It provides event-time processing with watermarks, large keyed state, and exactly-once state consistency through asynchronous distributed snapshots [Carbone et al. 2015a]. *(§29.9.1)*

Architectural decision — A design choice that addresses an architecturally significant requirement — one that affects structure, quality attributes, interfaces or technology in a way that is costly to change later. Examples: choosing a relational database over a key-value store; storing files in object storage; making photo processing asynchronous. *(§1.8.1)*

Architecture Decision Record (ADR) — A short document that captures one architectural decision: its context, the decision itself, its status, and its consequences. ADRs are numbered, kept in sequence (often in the code repository), and never edited to reverse a decision — a new ADR *supersedes* the old one [Nygard 2011]. *(§1.8.1)*

Architecture documentation — The written and diagrammed record of a system's structure and of the decisions that shaped it. It is written so that people who were not in the room (new team members, reviewers, your future self) can understand *what* the system is and *why* it is that way. *(§2.12.1)*

Asymmetric (public-key) cryptography — Cryptography using a mathematically linked **key pair**: a **public key** that can be shared with anyone and a **private key** kept secret by its owner. What one key does, only the other can undo or verify. Security rests on problems believed hard to invert: factoring large integers (RSA) or computing discrete logarithms on elliptic curves (ECC). *(§10.4.3)*

Atomic commitment — A protocol by which a set of **participants** (nodes that each executed part of a transaction) reach a single outcome, commit or abort, such that: (1) all participants that decide reach the same decision; (2) a participant cannot reverse a decision once made; (3) commit is decided only if every participant voted yes; (4) if there are no failures and every participant voted yes, the decision is commit; and (5) once failures are repaired and no new ones occur for long enough, every participant eventually decides [Bernstein et al. 1987]. *(§28.3.1)*

Audit log — An append-only record of security-relevant events — who (actor), did what (action), to which resource, when, from where, and with what result — retained so that actions can be investigated and attributed. A log is **tamper-evident** if any modification, deletion or reordering of past entries can be detected. *(§10.12)*

Authenticated encryption with associated data (AEAD) — A symmetric scheme that provides confidentiality for the plaintext *and* integrity for both the ciphertext and optional unencrypted **associated data** (AAD), producing an authentication **tag** that decryption checks before releasing any plaintext. AES-GCM and ChaCha20-Poly1305 are AEAD schemes. *(§10.4.2)*

Authentication (AuthN) — Verifying *who* a client is. **Authorization (AuthZ)** is deciding *what* that verified identity may do (§8.12). AuthN failures return `401`; AuthZ failures return `403`. *(§8.11)*

Autocomplete (typeahead) — A service that, given the prefix a user has typed so far, returns a short ranked list (typically 5–10) of likely completions, usually popular past queries or entity names, on every keystroke or after a short pause. *(§16.9.1)*

Autoscaling — Automatically changing the number of instances of a service in response to a measured or forecast signal, within configured minimum and maximum bounds. **Reactive** autoscaling responds to current metrics; **scheduled** autoscaling follows a calendar; **predictive** autoscaling acts on a forecast. *(§18.8.2)*

Availability — The fraction of time (or of requests) for which a system serves correctly: A = uptime/(uptime + downtime), or, request-based, A = successful requests/valid requests. It is commonly quoted in “nines”: 99.9% is “three nines”. *(§4.14.1)*

B

Back-of-the-envelope estimation — A quick calculation, done with round numbers and stated assumptions, that gives the order of magnitude of a system’s load and resource needs (requests per second, storage, bandwidth, memory, number of nodes). The aim is to be within a factor of about 2–3 of the truth with a few minutes of arithmetic. This section introduces the method. Chapter 89 is the full handbook. *(§2.11.1)*

Backpressure — A flow-control mechanism in which a slower downstream component signals upstream components to reduce their sending rate, by blocking, by advertising available capacity, or by rejecting, so that work in progress stays bounded instead of piling up in unbounded buffers. *(§22.10)*

Backward and forward compatibility — **Backward compatibility**: newer code can read data written by older code. **Forward compatibility**: older code can read data written by newer code [Kleppmann 2017, ch. 4]. *(§8.9.3)*

Batch processing — Running a computation over a **bounded** dataset (one whose size is known when the job starts, such as “yesterday’s logs”) that reads immutable input files, writes new output files, and is judged mainly by **throughput** (bytes or records processed

per second) and total completion time rather than by the latency of individual records. *(§29.3.1)*

BGP — A **path-vector** routing protocol in which neighboring routers (peers) exchange, over a TCP session on port 179, announcements of the form "prefix P is reachable via AS path [X, Y, Z]". Each AS chooses among the paths it hears according to its own **policy**, then re-announces its chosen path (with its own ASN prepended) to the peers its policy allows. *(§7.5.2)*

Bit and byte — A **bit** (symbol b) is a single binary digit, 0 or 1. A **byte** (symbol B) is 8 bits. Storage and memory are conventionally measured in bytes; network link speeds are conventionally measured in bits per second (bps). *(§4.5.1)*

Block, file and object storage — **Block storage** exposes a numbered array of fixed-size blocks (typically 512 B or 4 KiB) that a client reads and writes by offset; the client's own file system or database gives the blocks meaning. **File storage** exposes a hierarchical namespace of directories and byte-addressable files with POSIX-like operations (open, read at offset, write at offset, rename, lock), shared by many clients over a protocol such as NFS or SMB. **Object storage** exposes a flat namespace of immutable **objects**, each a byte string plus metadata addressed by a key within a **bucket**, accessed over HTTP with whole-object PUT, GET (optionally a byte range), DELETE and LIST. *(§15.3.1)*

Bloom filter — A probabilistic data structure for set membership, introduced by Burton Bloom in 1970, that stores a set of n items in an array of m bits using k hash functions [Bloom 1970]. A query answers either "definitely not in the set" or "probably in the set". It never gives a **false negative** (saying "no" for an item that was added), and gives a **false positive** (saying "yes" for an item that was not added) with a probability p that depends on m, n and k. *(§17.4.1)*

Bottleneck — The resource whose capacity limits the throughput of the whole system for a given workload. Throughput cannot rise above the bottleneck's capacity no matter how much the other resources are improved. Formally, it is the resource with the highest **service demand** per request (defined below). *(§2.6.1)*

Branch prediction — Hardware that guesses the outcome of a conditional jump (a **branch**, e.g. an `if` or a loop condition) before it is computed, so that the pipeline can keep fetching instructions. A wrong guess is a **misprediction** and forces a pipeline flush. *(§5.3.4)*

B-tree and B+ tree — A **B-tree** is a balanced search tree whose nodes are disk pages holding many sorted keys; every leaf is at the same depth, and nodes split and merge to stay between half-full and full [Bayer and McCreight 1972]. A **B+ tree** is the variant used by almost every relational database: internal nodes hold only separator keys and child pointers, all keys and values (or row pointers) live in the leaves, and leaves are linked to their siblings so range scans can walk sideways [Comer 1979; Petrov 2019]. *(§11.8.1)*

B-tree storage engine — A storage engine that keeps each table or index as a B+ tree of fixed-size pages and **updates pages in place**: a change to a key rewrites the page that holds it. Internal pages hold separator keys and child page numbers; leaf pages hold the keys with either the rows themselves (a clustered index) or pointers to them. *(§13.6.1)*

Bucket, key, object and flat namespace — A **bucket** is a named container with its own access policy, region and configuration. An **object** is an immutable byte string (from zero bytes to tens of terabytes) plus system metadata (size, content type, checksum, version ID) and user metadata. A **key** is the object's full name within the bucket, for example `photos/2026/10/u42/abc.jpg`. The namespace is **flat**: the slashes are ordinary characters, and "folders" are an illusion produced by listing keys that share a **prefix** and grouping them at a **delimiter** (usually `/`). *(§15.4.1)*

Buffer pool — A region of memory, managed by the database itself, divided into page-sized **frames** that cache disk pages. A **page table** maps page IDs to frames; each frame carries a **pin count** (how many threads are using it right now), a **dirty flag** (whether it differs from the disk copy) and bookkeeping for the **replacement policy** that chooses which page to evict. *(§13.4.1)*

Bulkhead — Partitioning a service's limited resources (threads, connections, memory, instances) into isolated pools, one per dependency, tenant or workload class, so that exhaustion of one pool cannot starve the others. *(§22.14)*

C

Cache — A store that holds copies of data whose authoritative version lives elsewhere (the **backing store** or **source of truth**), placed so that reading the copy is faster or cheaper than reading the original. A lookup that finds the data is a **hit**; one that does not is a **miss**. The **cache hit ratio** h is hits divided by lookups; the **miss ratio** is $1 - h$. *(§20.3.1)*

Cache-aside — The application reads the cache first; on a miss it reads the database and populates the cache itself. On a write it updates the database and then **deletes** (invalidates) the cache entry. The cache never talks to the database. Also called **look-aside** or **demand-filled** caching. *(§20.7.1)*

Cache avalanche — A large fraction of cache entries becomes unavailable at once, because many keys expire together or because cache nodes fail or restart, so the database suddenly receives the full read load. *(§20.13)*

Cache coherence — The guarantee that all cores observe a single, consistent sequence of values for each memory location, even though each core keeps private copies in its own caches. *(§5.4.6)*

Cache key — The value a cache uses to decide whether two requests may receive the same stored response. By default it is the method plus the full URL (scheme, host, path and query string), extended by any request headers listed in the response's `Vary` header. *(§21.6.2)*

Cache line — The fixed-size unit in which data moves between memory and caches. On x86-64 and most ARM server cores a line is 64 bytes; some designs (for example Apple's M-series cores) use 128 bytes (as of 2024). A load of a single byte brings its whole line into the cache. *(§5.4.3)*

Cache penetration — Requests for keys that do not exist in the database. Because there is nothing to cache, every such request misses and reaches the database. A bug or an attacker iterating random IDs can push the database's full read load through the cache. *(§20.12)*

Cache stampede (thundering herd, dog-piling) — When a popular entry expires or is evicted, many concurrent requests miss at once and all recompute or reload it, multiplying load on the backend exactly when it is busiest. *(§20.11)*

Cache tiers — The **edge** tier is the set of PoPs that accept user connections. A **mid-tier** (also *parent*, *regional* or *upper-tier* cache) is a smaller set of larger caches that edges consult on a miss. An **origin shield** is a mid-tier designated as the single path to a given origin. The **origin** is the customer's own infrastructure that holds the authoritative content. *(§21.4.1)*

Capacity planning — The recurring process of forecasting demand, translating it into the resources needed to serve it within the service-level objective with stated headroom and redundancy, and acquiring those resources before they are needed, allowing for the lead time to obtain them. *(§18.11.1)*

CAP theorem (Gilbert and Lynch) — In an asynchronous network model, where messages may be delayed arbitrarily and there are no clocks, it is impossible to implement a read/write data object that guarantees both of the following in all fair executions, including those in which messages are lost: **Consistency (C):** the object is **atomic**, that is, linearizable (§25.3). **Availability (A):** every request received by a non-failing node must eventually result in a response. **Partition tolerance (P)** is not a property the system chooses; it is the assumption that the network may lose arbitrarily many messages between nodes. *(§25.8.1)*

Causal consistency — Writes that are **causally related** must be seen by every client in their causal order; writes that are **concurrent** (neither happens before the other) may be seen in different orders by different clients. Formally, each client's view must be a legal sequential history that respects the causal order induced by program order and reads-from [Ahamad et al. 1995]. *(§25.5.1)*

Central processing unit (CPU) — The hardware that executes a program's **instructions**: simple operations such as "add these two numbers", "load a value from memory address X", or "if this value is zero, jump to instruction Y". A modern CPU package contains several independent **cores**, each of which can execute its own stream of instructions. *(§5.3.1)*

Certificate Transparency (CT) — A system of public, append-only, Merkle-tree-based logs into which CAs record every certificate they issue, so that mis-issued certificates become publicly detectable [RFC 6962]. *(§10.5)*

Chain replication — Replicas are arranged in a linear **chain**. Every update enters at the **head** and is forwarded node by node to the **tail**; the tail replies to the client. Every query is answered by the tail. A separate, fault-tolerant **master** service detects failures and reconfigures the chain [van Renesse and Schneider 2004]. *(§27.5.1)*

Chubby — Google's lock service, providing coarse-grained advisory locks and small files in a file-system-like namespace, replicated across a five-node **cell** with Paxos, and used chiefly for leader election, naming and storing small amounts of configuration [Burrows 2006]. *(§26.14.1)*

CIDR block — An address range written as `address/prefix-length`, for example `192.168.10.64/26`. The prefix length n says that the first n bits identify the network; the remaining $32 - n$ bits (IPv4) or $128 - n$ bits (IPv6) identify hosts. The **subnet mask** is the same information as a dotted number with n one-bits followed by zeros (`/26` = `255.255.255.192`). *(§7.4.2)*

Circuit breaker — A client-side proxy for calls to a dependency that tracks recent failures and, when they cross a threshold, **opens**: it fails calls immediately without contacting the dependency for a cool-down period, then lets a few trial calls through (**half-open**) and **closes** again only if they succeed. *(§22.13)*

Client–server architecture — A way of dividing a system into two roles: a **client**, which initiates requests on behalf of a user or another program, and a **server**, which waits for requests, performs work on shared resources (data, computation) and returns responses. The two communicate over a network using an agreed **protocol**. *(§8.3.1)*

Client-side load balancing — The calling process obtains the list of backend endpoints from service discovery (§34.7) and chooses a backend for each request itself, with no proxy in the path. A **lookaside** balancer is an external service that tells clients *which* backends to use (and with what weights) while the traffic still flows directly from client to backend. *(§19.10)*

Clock rate — The frequency at which a core's internal circuitry steps from one state to the next, measured in hertz (cycles per second). A 4 GHz core has a **cycle time** of $1 / (4 \times 10^9)$ s = 0.25 ns. *(§5.3.2)*

Clustered and secondary index — In a **clustered index** (an index-organized table), the leaf level of the index *is* the table: rows are stored in key order inside the index's leaf pages. A table can have at most one. A **secondary index** stores, for each key, a pointer to the row; the table can have many. *(§11.7.2)*

Columnar storage (column store) — A layout that stores each column of a table contiguously — all values of `price`, then all values of `country` — instead of storing each row's fields together. Column stores are optimized for analytical queries that read a few columns of many rows [Abadi et al. 2013]. *(§13.13.1)*

Compaction — The background merge of sorted runs in an LSM tree that removes overwritten values and tombstones and bounds the number of runs a read must consult. A **compaction strategy** decides which runs to merge and when; it sets the engine's read, write and space amplification. *(§13.9.1)*

Compare-and-swap (CAS) — An atomic CPU instruction that, in one indivisible step, compares a memory location with an expected value and, only if they are equal, replaces it with a new value, reporting whether it succeeded. On x86-64 it is `LOCK CMPXCHG`; ARMv8.1 and later provide `CAS`, and earlier ARM uses load-linked/store-conditional pairs. *(§6.10.4)*

Component — A unit of a system with a clear responsibility and an interface — for example a web service, a database, a cache, a message queue, a load balancer or a third-party payment provider. *(§1.3.1)*

Components of network delay — **Propagation delay**: time for a signal to travel the physical distance. **Transmission (serialization) delay**: time to push all of a packet's bits onto the link. **Queuing delay**: time spent waiting in buffers behind other packets. **Processing delay**: time for a device to examine headers, look up routes and copy the packet. *(§7.15.1)*

Concept-mastery loop — A fixed sequence of ten learning activities applied to one system-design concept. Each activity produces a concrete artifact (a definition, a diagram, a program, a design). The concept counts as mastered only when you can rebuild every artifact without notes. *(§3.3.1)*

Concurrency and parallelism — **Concurrency** is the *structuring* of a program as multiple tasks whose lifetimes overlap, so that progress on one does not wait for the others to finish. **Parallelism** is the *simultaneous execution* of multiple computations on multiple hardware units (cores, CPUs, GPUs). *(§6.8)*

Condition variable — A queue of threads waiting for some condition on shared state to become true, always used together with a mutex. `wait` atomically releases the mutex and sleeps; `notify` (`signal`) wakes one waiter and `notify_all` (`broadcast`) wakes all. *(§6.10.3)*

Conflict-free replicated data type (CRDT) — A data type whose replicas can be updated independently and concurrently without coordination and are guaranteed to converge to the same state once they have received the same updates, a property called **strong eventual consistency** [Shapiro et al. 2011a]. *(§27.14.3)*

Congestion window (cwnd) — A sender-side limit, in bytes or segments, on unacknowledged data in flight, based on the sender's estimate of what the network path can absorb. The sender may have at most $\min(\text{cwnd}, \text{rwnd})$ bytes outstanding. *(§7.7.7)*

Connection gateway — A dedicated, thin service tier whose only job is to terminate long-lived client connections (WebSocket, SSE, long polling), authenticate them, track which user and topics each connection belongs to, and move messages between those connections and back-end services through a messaging layer. It contains no business logic. *(§9.9.1)*

Connection pooler — A component that keeps a few open database connections and lends them to many clients per session, transaction or statement, either inside the application or as a shared proxy. *(§14.13.1)*

Consensus — A set of processes, each of which may **propose** a value, must **decide** on a single value such that: **agreement** — no two correct processes decide differently; **validity** (integrity) — the decided value was proposed by some process; **termination** — every correct process eventually decides. Once decided, a value never changes. *(§26.3.1)*

Consistent hashing — A way of assigning keys to nodes so that, when a node is added or removed, only about K/N of the K keys change owner, where N is the number of nodes. Both keys and nodes are hashed onto the same circular number space (the **ring**). Each key belongs to the first node found by moving clockwise from the key's position [Karger et al. 1997]. *(§3.3.2)*

Consistent hashing with bounded loads — A variant of consistent hashing in which every backend has a **capacity** of $\lceil (1+\varepsilon)\,m/n \rceil$ for m current items (requests, sessions) and n backends. An item goes to the first backend clockwise from its hash that is below capacity, so overflow from a full backend walks on to the next [Mirrokni, Thorup & Zadimoghaddam 2018]. *(§19.4.7)*

Content-defined chunking (CDC) — Splitting a byte stream into variable-size chunks whose boundaries are chosen by the content itself: a rolling hash is computed over a small sliding window, and a boundary is declared wherever the hash matches a pattern (for example, its low 13 bits equal a fixed value). Each chunk is then identified by a cryptographic hash of its bytes, so identical chunks are stored once (**deduplication**). *(§15.8.3)*

Content delivery network (CDN) — A distributed system of caching reverse proxies (§7.12) deployed in many network locations, which serves content on behalf of an **origin**

(the authoritative server for the content) from the location nearest each user, fetching from the origin only on a cache miss. *(§21.3.1)*

Context diagram — A diagram that shows the whole system as a single box, surrounded by the people (actors) and external systems that interact with it, with each interaction labeled. Context diagrams come from structured analysis [DeMarco 1979] and are Level 1 of the C4 model [Brown C4]. *(§2.4.1)*

Context switch — The kernel operation that stops one thread running on a CPU core, saves its CPU state into its PCB, and loads the saved state of another thread so that it resumes where it left off. *(§6.4)*

Coordinated omission — A measurement error in which the load generator, by waiting for slow responses before sending more requests, fails to send (and therefore fails to measure) the requests that would have experienced the slowness. The tester and the system "coordinate" to omit the worst samples. The term and diagnosis are due to Gil Tene [Tene, How NOT to Measure Latency]. *(§23.5.3)*

Cost-based optimizer — The component that enumerates equivalent physical plans for a query (access paths, join orders, join algorithms), estimates each plan's cost from **statistics** about the data using a **cost model**, and returns the plan with the lowest estimated cost. *(§11.10.1)*

Count-Min sketch — A $d \times w$ array of counters with one hash function per row that estimates the frequency of any item in a stream. Each update increments one counter per row; each query returns the minimum of the item's d counters. The estimate never underestimates and, with probability at least $1 - \delta$, overestimates by at most εN, where N is the total of all updates [Cormode & Muthukrishnan 2005]. *(§17.6.1)*

CPU-bound and I/O-bound — A workload is **CPU-bound** when its speed is limited by how fast the processor can execute instructions; giving it a faster CPU or more cores speeds it up. It is **I/O-bound** when it spends most of its time waiting for storage or the network; a faster CPU barely helps, but more concurrency or faster devices do. A related category, **memory-bound**, is limited by memory latency or bandwidth (§5.9) — the CPU looks "busy" but is mostly stalled. *(§5.8)*

CPU cache — A small, fast memory built from SRAM (static RAM) on the processor chip that holds copies of recently used main-memory data, so that most loads and stores do not have to wait for DRAM. *(§5.4.1)*

CPU scheduler — The part of the kernel that decides which ready thread runs next on each core and for how long. The rule it follows is the **scheduling policy**; the mechanism that performs the switch is the dispatcher (§6.4). *(§6.5)*

Critical section — A region of code that accesses shared state and must not be executed by more than one thread at a time. Guaranteeing this is **mutual exclusion**. *(§6.9)*

Cryptographic hash function — A deterministic function H that maps an input of any length to a fixed-length output (the **digest**), such that it is computationally infeasible to (1) find an input that hashes to a given digest (**preimage resistance**), (2) given one input, find a different input with the same digest (**second-preimage resistance**), and (3) find any two distinct inputs with the same digest (**collision resistance**). *(§10.4.1)*

Cuckoo filter — A hash table of b-slot buckets that stores a short **fingerprint** of each item (an f-bit hash) in one of two candidate buckets. Membership checks two buckets for a matching fingerprint; deletion removes one copy of the fingerprint. *(§17.4.7)*

D

DataFrame, Dataset and Catalyst — A **DataFrame** is a distributed table with a known schema (named, typed columns), manipulated with relational operators or SQL. A **Dataset** (Scala and Java) adds compile-time types to the same abstraction; in Python and R, every DataFrame is untyped at compile time. **Catalyst** is Spark SQL's extensible query optimizer, which turns DataFrame or SQL queries into an optimized physical plan using rule-based and cost-based transformations of trees [Armbrust et al. 2015]. *(§29.5.4)*

Data locality — Scheduling a task on or near the node that stores its input, so that the input is read from local disk or memory instead of over the network. Schedulers rank placements as **process-local** (data cached in the same executor), **node-local**, **rack-local** and **off-rack** (any). *(§29.7)*

Deadline — An absolute point in time by which a client needs an RPC to complete. gRPC transmits it as a remaining timeout; when it passes, the client stops waiting with `DEADLINE_EXCEEDED` and the server can see that the call was canceled [gRPC Docs: Deadlines]. *(§8.8.4)*

Deadlock — A state in which each thread in a set is waiting for a resource held by another thread in the same set, so none can ever proceed. *(§6.11.1)*

Decomposition — Dividing a system into smaller parts (**components**) so that each part has one clear responsibility, a well-defined interface, and as few dependencies on other parts as possible. Each part can then be understood, built, scaled and changed mostly on its own. *(§2.3.2)*

Deliberate practice — Effortful practice designed to improve performance. It targets specific weaknesses, has clear goals, gives immediate feedback and is usually guided by a teacher or coach [Ericsson et al. 1993]. *(§3.4.6)*

Denormalization — Deliberately storing a fact more than once, or storing a derived value, to make important reads cheaper, while accepting the cost of keeping the copies consistent. *(§11.5.5)*

Design assumption — A belief about the workload, the environment or the business that the design depends on but that has not yet been verified, for example “80% of requests are reads” or “the payment provider responds within 1 s at p99”. An assumption is **stated** when it is written down with its value, its source, your confidence in it and how it will be checked. *(§2.7.1)*

Design document (design doc) — A written proposal, produced before significant implementation, that states a problem, its goals and non-goals, a proposed design, the alternatives considered, and cross-cutting concerns (security, privacy, operations, cost), so that others can review it while changes are still cheap. *(§1.10.1)*

Designing for failure — Treating component failure as a *normal, expected event* rather than an exception. For every component and dependency, the design states what happens when it is slow, returns errors or disappears, and makes sure the **blast radius** (the set of users and features affected) is as small as possible. *(§2.9.1)*

Deterministic database — A database in which replicas and partitions first agree on a global order of transaction *inputs*, then execute transactions so that the outcome depends only on that order and the initial state. Every node that executes the same log reaches the same state, so no node needs to ask another whether to commit [Thomson et al. 2012]. *(§28.9.1)*

Digital signature — A scheme with three algorithms: key generation, **sign** (using the private key, produces a signature over a message) and **verify** (using the public key, checks a signature against a message). A valid signature shows that the message was approved by the holder of the private key and has not changed since. Because verification needs only the public key, *anyone* can verify, which provides non-repudiation. *(§10.4.5)*

Direct memory access (DMA) — A mechanism that lets a device controller read from or write to main memory directly, without the CPU copying each word; the CPU only sets up the transfer and is told when it finishes. *(§5.7.3)*

Direct server return (DSR) — A forwarding mode in which the balancer delivers request packets to the backend but the backend sends responses **directly to the client**, using the VIP as its source address, bypassing the balancer on the return path. *(§19.3.1)*

Distributed denial of service (DDoS) — An attack that makes a service unavailable by overwhelming some finite resource — network bandwidth, packet-processing capacity, connection-state tables, or application CPU, memory and database capacity — with traffic from many sources at once, typically a **botnet** of compromised devices or abused open servers. *(§10.11)*

Distributed file system — A file system whose namespace and data are spread across many machines, so that files can be larger than any one disk and throughput can grow with the number of machines, while clients still use file-like operations (create, open, read, write or append, delete). *(§15.5.1)*

Distributed system — A set of independent computers (**nodes**) that have no shared memory and no shared clock, communicate only by exchanging messages over a network, and cooperate to provide a service. Van Steen and Tanenbaum describe it from the user's side as "a collection of autonomous computing elements that appears to its users as a single coherent system" [van Steen and Tanenbaum 2017]. *(§24.3.1)*

DNS time to live (TTL) — A per-record value, in seconds, that tells any cache how long it may reuse the record before asking again. **Negative caching** stores "this name does not exist" (NXDOMAIN) answers for a duration derived from the zone's SOA record [RFC 2308]. *(§7.6.4)*

Document database — A database that stores self-describing, hierarchical records — **documents**, usually JSON or a binary encoding of it such as BSON — grouped into **collections**, and that can index and query fields inside the documents, including nested fields and array elements. *(§12.5.1)*

Domain Name System (DNS) — A hierarchical, distributed database that maps human-readable **domain names** (such as `www.example.com`) to data, most often IP addresses, and the protocol used to query it [RFC 1034; RFC 1035]. *(§7.6.1)*

DRAM (dynamic random-access memory) — The technology used for main memory. Each bit is stored as charge in a tiny capacitor that leaks and must be **refreshed** every few tens of milliseconds. DRAM is **volatile**: its contents are lost when power is removed. *(§5.5.1)*

Durability — The probability that data, once acknowledged as written, is not lost over a given period (usually a year). It is quoted in nines like availability: "eleven nines of durability" means an annual loss probability of 10^{-11} per object. *(§4.16.1)*

Dynamic site acceleration (DSA) — Routing uncacheable requests through the CDN anyway, so that TCP and TLS terminate near the user and the edge forwards each request to the origin over pre-established, tuned, long-lived connections, sometimes along an overlay path chosen from measurements. *(§21.7.2)*

E

Elasticsearch cluster — A set of nodes that share one cluster state (index definitions, mappings and the location of every shard copy). An **index** is a logical collection of docu-

ments divided into **primary shards**; each primary has zero or more **replica shards**; every shard is one Lucene index. *(§16.8.1)*

Encapsulation — The process by which each layer of the stack prepends its own header (and sometimes appends a trailer) to the data unit handed down by the layer above, treating that data unit as an opaque payload. **Decapsulation** is the reverse at the receiver. *(§7.3.1)*

Envelope encryption — Encrypting data with a **data-encryption key (DEK)** generated for that data, then encrypting ("wrapping") the DEK with a KEK held in a KMS, and storing the wrapped DEK alongside the ciphertext [AWS Docs: KMS concepts]. *(§10.9.2)*

Erasure code and RS(k, m) — An **erasure code** transforms k data fragments into $k + m$ fragments such that the original data can be rebuilt from some subset of them after the others are lost ("erased"). A **Reed–Solomon** code RS(k, m) is a **maximum distance separable (MDS)** code: *any* k of the $k + m$ fragments suffice, so it tolerates the loss of any m fragments. A code is **systematic** when the first k fragments are the unmodified data, so healthy reads need no decoding. *(§15.6.1)*

Event loop — A single thread that repeatedly waits for events (ready sockets, expired timers, completed tasks) using an I/O multiplexing call, and dispatches each event to a short, non-blocking handler. *(§6.13.4)*

Eventual consistency — If no new updates are made to an object, eventually all reads return the last updated value [Vogels 2009]. Equivalently, replicas that have received the same set of updates **converge** to the same state. **Strong eventual consistency** strengthens this: replicas that have received the same updates are in the same state immediately, with no further coordination, which is what conflict-free replicated data types (CRDTs) guarantee (§27.14). *(§25.7.1)*

Expected value — The probability-weighted average outcome of a random quantity X: $E[X] = \sum_i x_i \, P(X = x_i)$. It is the long-run average over many repetitions. *(§4.6.3)*

Exponent — In b^n, the **base** b is multiplied by itself n times; n is the **exponent** (or power). By convention $b^0 = 1$ and $b^{-n} = 1/b^n$. *(§4.3.1)*

External consistency — If transaction T_1 commits before T_2 starts, in absolute (wall-clock) time, then T_1's commit timestamp is smaller than T_2's. Combined with serializability, this is **strict serializability**: the serial order respects real-time order, as linearizability does for single objects (§25.3). *(§28.8.3)*

F

Failover — The sequence that replaces a failed node in a role: **detection**, **election** of a replacement, **reconfiguration** (making the replacement consistent and fencing the old node) and **client redirection**. *(§27.11.1)*

Failure detector — A component that outputs, for each monitored process, whether it currently **suspects** that process has crashed. It is **complete** if every crashed process is eventually suspected and **accurate** if live processes are not suspected. In an asynchronous network no detector can be both perfectly complete and perfectly accurate, because a slow process is indistinguishable from a dead one [Chandra and Toueg 1996]. *(§27.10.1)*

False sharing — A performance problem in which threads on different cores write to *different* variables that happen to sit in the *same* cache line, so the coherence protocol bounces the line between cores as if they were sharing data. *(§5.4.7)*

Fencing token — A number, increasing with every grant of a lock or lease, that the holder attaches to every request to the protected resource. The resource remembers the highest token it has accepted and rejects any request carrying a lower one. *(§26.13)*

Firewall — A device or software function that permits or denies traffic according to rules. A **stateless** (packet-filter) firewall evaluates each packet independently against rules on addresses, ports and protocol. A **stateful** firewall tracks connections, so a reply to an allowed outbound connection is automatically permitted. **Application-layer firewalls**, including web application firewalls (WAFs), parse protocols such as HTTP to block malicious requests. *(§7.14)*

Fixed window counter — A limiter that divides time into aligned windows of length W (for example each clock minute), keeps one counter per key per window, and admits a request if the current window's counter is below the limit L. The counter resets at each window boundary. *(§22.6.1)*

FLP impossibility — In a fully asynchronous message-passing system with reliable links, no *deterministic* protocol can solve **consensus** (all correct processes decide the same value, the value was proposed by some process, and every correct process eventually decides) if even one process may crash-stop [Fischer et al. 1985]. *(§24.13.1)*

Forward secrecy — A property of a key-exchange protocol whereby compromise of a party's long-term private key does not reveal session keys from past sessions. It is achieved by using fresh **ephemeral** key pairs for every session (the "E" in ECDHE) and deleting them afterward. *(§10.4.4)*

Four golden signals — Google's site reliability engineering guidance says that if you can measure only four metrics of a user-facing system, measure **latency** (time to serve a request, tracking successful and failed requests separately), **traffic** (demand, such as re-

quests per second), **errors** (rate of failed requests, explicit or implicit) and **saturation** (how "full" the service is, emphasizing its most constrained resource) [Beyer et al. 2016, ch. 6]. *(§23.3.3)*

Freshness and validation — A stored response is **fresh** while its age is below its **freshness lifetime** and may then be served without contacting the origin; once **stale**, it must normally be **revalidated**, by sending a conditional request (`If-None-Match` with the stored `ETag` , or `If-Modified-Since`), to which the origin answers `304 Not Modified` (keep using the copy) or `200 OK` with a new body [RFC 9111; RFC 9110]. *(§21.6.1)*

Functional dependency — In a relation R, the functional dependency $X \to Y$ holds if any two rows that agree on the columns X also agree on the columns Y. X is the **determinant**. Example: `product_id → product_name` because one product ID always has one name. *(§11.5.1)*

Functional requirement (FR) — A statement of a behavior the system must provide: an input, action or event, and the observable result. Functional requirements describe *what* the system does — for example, "a signed-in user can upload a JPEG or PNG photo up to 20 MB, and it appears in their followers' feeds". *(§1.6.1)*

G

Garbage collection (GC) — Automatic reclamation of heap memory that the program can no longer reach. A **tracing** collector finds live objects by following references from **roots** (stacks, globals, registers) and reclaims everything else. A **stop-the-world (STW) pause** is an interval in which all application threads (the **mutator**) are halted so the collector can work on a consistent heap [Jones et al. 2023]. *(§23.9.1)*

Geohash — A string encoding of a latitude/longitude cell, published by Gustavo Niemeyer in 2008, formed by alternately bisecting the longitude and latitude ranges, interleaving the resulting bits (longitude first) and writing every 5 bits as one base-32 character [Niemeyer 2008]. Each additional character subdivides the cell into 32 smaller cells, so a geohash that is a prefix of another names a cell containing it. *(§17.9.2)*

Gossip protocol — A protocol in which, every round, each node exchanges state with f randomly chosen peers (the **fanout**). In **push** gossip informed nodes send updates; in **pull** gossip nodes ask peers for news; **push–pull** does both. **Anti-entropy** gossip exchanges full state; **rumor mongering** spreads only recent updates and stops after a while [Demers et al. 1987]. *(§27.10.3)*

Graph database — A database whose data model is a graph of **vertices** (nodes) and **edges** (relationships), optimized for traversing edges. In the **property graph** model, nodes carry labels, edges carry a type and a direction, and both carry key-value properties. In the **RDF** model, all data is a set of subject–predicate–object **triples**. *(§12.7.1)*

GraphQL — A query language for APIs and a runtime for executing those queries against a strongly typed **schema**. The client sends a document describing exactly the fields it wants; the server returns JSON of the same shape [GraphQL Spec]. *(§8.7.1)*

Gray failure — A component failure that is *differentially observable*: some observers (often the system's own failure detector) consider the component healthy, while others (often the applications using it) experience it as failed or badly degraded. *(§24.8.2)*

gRPC — An open-source RPC framework, originally developed at Google and released in 2015, that uses Protocol Buffers as its interface definition language and message format and HTTP/2 as its transport [gRPC Docs]. *(§8.8.2)*

H

Happened-before (→) — The smallest relation on events such that: (1) if a and b occur in the same process and a comes first, $a \to b$; (2) if a is the sending of a message and b its receipt, $a \to b$; (3) if $a \to b$ and $b \to c$, then $a \to c$. Two distinct events are **concurrent** ($a \parallel b$) if neither $a \to b$ nor $b \to a$. *(§24.10.1)*

Hard disk drive (HDD) — A storage device that records data magnetically on spinning platters, read and written by heads mounted on a moving arm (the **actuator**). Data is organized in concentric **tracks** divided into **sectors** of 512 or 4096 bytes. *(§5.6.1)*

Hash function — A deterministic function h that maps an input of any length to a fixed-size integer (for example 32, 64 or 128 bits), called the **hash** or **digest**. A **non-cryptographic hash function** is designed for speed and statistical uniformity on ordinary inputs. A **cryptographic hash function** additionally resists an adversary who tries to find collisions or preimages (§10.4.1). *(§17.3.1)*

Hash slot (Redis Cluster) — One of 16,384 buckets into which Redis Cluster divides the keyspace: `slot = CRC16(key) mod 16384`. Each primary owns a set of slots. If a key contains `{...}`, only the substring inside the braces (a **hash tag**) is hashed, so related keys can be forced into one slot [Redis Docs: Cluster specification]. *(§20.6.4)*

Health check — A mechanism by which a balancer decides whether a backend should receive traffic. An **active** check sends synthetic probes (a TCP connect, an HTTP `GET /healthz`, a gRPC health RPC) on a timer. A **passive** check, also called **outlier detection**, watches real traffic for errors, timeouts or latency and **ejects** misbehaving backends for a period. *(§19.5.1)*

Hedged request — A request sent to one replica and, if no response arrives within a **hedge delay** d, sent again to a second replica; the client uses whichever response arrives first and cancels or ignores the other [Dean and Barroso 2013]. *(§23.7.1)*

High-level design (HLD) — A description of a system as a set of major components (services, data stores, queues, caches, external dependencies), their responsibilities, the interfaces and protocols between them, data flows, and deployment topology — at a level of detail where each component is a box whose internals are not yet specified. *(§1.5.1)*

Horizontal scaling (scaling out) — Increasing capacity by adding more nodes that share the workload, for example by running 20 identical application servers behind a load balancer instead of one. Removing nodes when load falls is **scaling in**. *(§18.4.1)*

Hot key — A single key whose request rate exceeds what the node holding it can serve, so partitioning cannot help: all requests for one key go to one node. *(§20.15)*

Hot partition and hot key — A **hot partition** receives enough excess requests or data to saturate its node while peers idle. A **hot key** is a single key causing that excess. **Skew** is uneven distribution of data or load in general. *(§14.10.1)*

HTTP adaptive streaming — Video is encoded at several quality levels (a **bitrate ladder**), each cut into **segments** of a few seconds stored as ordinary files. A **manifest** lists the levels and segment URLs. The player downloads the manifest, then fetches segments one by one over HTTP, choosing a level for each segment. *(§21.8.1)*

HTTP cookie — A small name–value pair that a server asks the browser to store, via the `Set-Cookie` response header, and that the browser automatically returns in the `Cookie` request header on later requests to matching URLs [RFC 6265]. *(§8.10.1)*

Hybrid logical clock — A timestamp (l, c) where l is the largest physical clock value the node has seen (from its own clock or from any message) and c is a logical counter that orders events sharing the same l. Timestamps compare lexicographically: first by l, then by c. *(§24.12.1)*

HyperLogLog (HLL) — A probabilistic algorithm that estimates the number of distinct items (the **cardinality**) in a multiset using m small registers, with relative standard error about $1.04/\sqrt{m}$, independent of the cardinality itself [Flajolet et al. 2007]. *(§17.5.1)*

I

Idempotency key — A unique value, generated by the client once per *logical* operation and sent with every attempt of that operation (for example in an `Idempotency-Key` header), that the server uses to recognize retries: the first attempt executes and its result is stored under the key; later attempts with the same key return the stored result instead of executing again. *(§22.15.1)*

Idempotent method — A method for which the intended effect on the server of sending the same request $N > 0$ times is the same as sending it once [RFC 9110 §9.2.2]. The *re-*

sponse may differ (a second `DELETE` may return 404); the *state* does not. *(§8.6.3)*

Index — A secondary data structure, kept in sync with a table by the database, that maps values of one or more **key columns** to the locations of the rows holding them, so that rows can be found without scanning the whole table. Most relational indexes are B+ trees (§11.8); others include hash, GiST, GIN and BRIN indexes in PostgreSQL and full-text and spatial indexes in MySQL. *(§11.7.1)*

In-doubt (blocked) transaction — A transaction at a participant that has voted yes but has not learned the outcome. It must hold its locks and cannot decide until the coordinator, or another participant that knows the outcome, answers. *(§28.4.3)*

Instruction pipeline — A technique in which the core splits instruction execution into stages and works on different stages of several consecutive instructions at the same time, like an assembly line. *(§5.3.3)*

Interleaving — Mixing different kinds of problems or concepts within one practice session (A B C A C B), instead of finishing all practice on one kind before moving to the next (AAA BBB CCC, called **blocking**). *(§3.4.5)*

Inverted index — A mapping from each distinct **term** (a normalized word or token) to a **postings list**: the sorted list of identifiers of the documents that contain the term, optionally with the term's frequency and positions in each document. The set of all terms is the **dictionary** (or **vocabulary**, or **lexicon**). *(§16.3.1)*

I/O (input/output) — Any transfer of data between the CPU/memory and a device outside it: storage, network cards, GPUs, keyboards. A **device controller** on each device does the physical work; the CPU communicates with it through **device registers** and shared memory. *(§5.7)*

IOPS — Input/output operations per second: the number of independent read or write requests a device completes per second, quoted for a stated request size (usually 4 KiB) and **queue depth** (number of requests outstanding at once). *(§5.6.5)*

IP address — A numeric identifier assigned to a network interface that the Internet Protocol uses to deliver packets. An **IPv4** address is 32 bits, written as four decimal octets (`203.0.113.25`) [RFC 791]. An **IPv6** address is 128 bits, written as eight groups of four hexadecimal digits separated by colons (`2001:db8:0:1::25`) [RFC 8200]. *(§7.4.1)*

Isolation anomalies — **Dirty write:** a transaction overwrites another's uncommitted write. **Dirty read:** a transaction reads another's uncommitted write (which may later roll back). **Non-repeatable (fuzzy) read:** a transaction reads a row twice and sees different committed values because another transaction updated it in between. **Phantom:** a transaction runs the same *predicate* query twice (`WHERE status = 'pending'`) and gets a different set of rows because another transaction inserted or deleted matching rows. **Lost up-**

date: two transactions read the same value, both compute a new value from it and write; the second write silently overwrites the first. **Read skew:** a transaction sees part of another transaction's effects (reads account A before a transfer commits and account B after it). **Write skew:** two transactions read an overlapping set of rows, each checks an invariant, then each writes a *different* row; each write is valid alone but together they violate the invariant. *(§11.13.1)*

J

JSON Web Token (JWT) — A compact, URL-safe string representing a set of **claims** (statements about a subject) as a JSON object, digitally signed (JWS) or encrypted (JWE) so its integrity can be verified [RFC 7519]. This section covers the signed form, which is by far the most common. *(§8.13.1)*

K

Key management service (KMS) — A managed service, backed by hardware security modules (HSMs), that generates and stores **key-encryption keys (KEKs)** that never leave it in plaintext, performs cryptographic operations with them on request, enforces access policies on every use and logs every call. Examples: AWS KMS, Google Cloud KMS, Azure Key Vault, HashiCorp Vault's Transit engine. *(§10.9.2)*

Keys — A **superkey** is a set of columns whose values are unique across all rows. A **candidate key** is a minimal superkey: remove any column and it stops being unique. The **primary key** is the candidate key chosen as the row's main identifier; it must be unique and NOT NULL. Other candidate keys are enforced with `UNIQUE` constraints. A **foreign key** is a column (or set) in one table whose values must match a key of another table, a rule called **referential integrity**. *(§11.4.1)*

Key-value store — A database whose primary interface is `get(key)`, `put(key, value)` and `delete(key)`, where the value is opaque to the database or has a small set of built-in types. The key alone determines where the value lives. *(§12.4.1)*

L

Lambda and Kappa architectures — The **Lambda architecture** runs two parallel paths over the same immutable master dataset: a **batch layer** that periodically recomputes complete, accurate views from all history, and a **speed layer** that processes recent data incrementally with low latency; a **serving layer** merges the two at query time [Marz and Warren 2015]. The **Kappa architecture** uses a single stream-processing path over a

replayable log; to change logic or fix a bug, it reprocesses history by replaying the log through a new version of the job [Kreps 2014]. *(§29.10)*

Lamport clock — Each process P_i keeps an integer counter C_i, initially 0. **Rule 1:** before each event (local, send or receive), $C_i \leftarrow C_i + 1$; a send attaches the new C_i to the message. **Rule 2:** on receiving a message with timestamp t_m, set $C_i \leftarrow \max(C_i, t_m)$, then apply rule 1. The event's timestamp is the resulting C_i. *(§24.10.2)*

Last-writer-wins (LWW) — Each write carries a timestamp, and when replicas hold different values for an item, the one with the greatest timestamp is kept and the others are discarded. *(§27.14.1)*

Latency and bandwidth (hardware) — **Latency** is the time from issuing one request to receiving its first byte (for example, ~100 ns for a DRAM load). **Bandwidth** is the maximum rate at which data can be moved once transfers are flowing (for example, ~38 GB/s for one DDR5-4800 channel). Achieved bandwidth is **throughput**. *(§5.9.1)*

Layer 4 (L4) and layer 7 (L7) load balancing — An **L4 load balancer** makes its decision from transport-layer information (IP addresses, ports, protocol) and balances **connections or flows**: every packet of one TCP connection goes to the same backend. An **L7 load balancer** terminates the client connection, parses the application protocol (usually HTTP), and balances **requests**: two requests on one connection may go to different backends. *(§19.3.1)*

Leader election — A protocol by which the nodes of a group choose one node, the **leader**, to coordinate some activity (ordering writes, running a scheduler, owning a partition), and choose a replacement when it fails. The goal is *at most one* leader at a time (safety) and *eventually some* leader (liveness). *(§26.5.1)*

Leaderless replication — Any replica of a key may accept reads and writes; the client, or a **coordinator** node acting for it, sends each request to several replicas in parallel and treats it as successful once enough of them respond. There is no failover because there is no leader [DeCandia et al. 2007]. *(§27.7.1)*

Leaf–spine (folded Clos) network — A two-tier topology in which every **leaf** (top-of-rack) switch connects to every **spine** switch, and servers attach only to leaves. Any two servers on different leaves are exactly two switch hops apart (leaf → spine → leaf), and traffic is spread across all spines with ECMP. It is a form of the multistage **Clos network**, first described by Charles Clos in 1953 for telephone switching. Larger fabrics add a third tier. *(§7.17.1)*

Leaky bucket — (a) **As a queue:** requests enter a FIFO buffer of capacity Q that is drained at a constant rate r; arrivals that find the buffer full are dropped. Output is perfectly smooth. (b) **As a meter:** a counter (the "water level") increases by one per admitted request and leaks at rate r; a request that would overflow capacity is rejected immedi-

ately. The meter form makes the same admit/reject decisions as a token bucket with the same rate and capacity. *(§22.5.1)*

Lease — A time-bounded grant of a right (to cache data, to act as leader, to hold a lock) from a grantor to a holder. Within the term the holder may act without consulting the grantor; after the term, the right lapses automatically, even if the holder has crashed or is unreachable [Gray and Cheriton 1989]. *(§26.11.1)*

Least connections — Send each new connection or request to the backend with the fewest **active** (in-flight) connections or requests, as counted by the balancer. *Weighted least connections* divides each count by the backend's weight. Envoy calls the request-counting variant **least request**; AWS ALB calls it **least outstanding requests**. *(§19.4.2)*

Least response time — Choose the backend with the lowest recent latency, usually an **exponentially weighted moving average (EWMA)** of observed response times, often multiplied by the in-flight count so that both queue length and speed count. *(§19.4.3)*

Linearizability — A history is **linearizable** if each operation can be assigned a single instant, its **linearization point**, somewhere between its invocation and its response, such that (1) the operations, ordered by these points, form a **legal sequential history** of the object (every read returns the value of the most recent preceding write), and (2) therefore whenever a precedes b in real time, a comes before b in that order. Pending operations may be either included (with a linearization point after their invocation) or dropped. An object is linearizable if all its histories are [Herlihy & Wing 1990]. *(§25.3.2)*

Little's Law — For any stable system observed over a long period, the average number of items in the system equals the average arrival rate multiplied by the average time each item spends in the system: $L = \lambda W$ [Little 1961]. *(§4.11.1)*

Load balancer — A component that receives traffic addressed to a single service endpoint, the **virtual IP address (VIP)** or service name, and distributes it across a **pool** of **backends** (also called *upstreams*, *targets* or *real servers*) according to a **balancing algorithm**, while excluding backends that are unhealthy. *(§19.3.1)*

Load shedding and throttling — **Load shedding** is a server rejecting some requests early and cheaply (usually with 503) because accepting them would push it past capacity, regardless of whether each client is within its quota. **Throttling** is slowing a client down, by delaying or rejecting its requests; rate limiting is the quota-based form. **Goodput** is the rate of requests completed *usefully*: correctly and before the client gave up. *(§22.9.1)*

Local and global secondary indexes — A **local** (document-partitioned) index lives in each partition and covers only its rows. A **global** (term-partitioned) index covers all rows and is partitioned by the *indexed value*, so all entries for one value live together [Kleppmann 2017]. *(§14.11.1)*

Logarithm — The logarithm base b of x, written $\log_b x$, is the exponent to which b must be raised to give x: if $b^y = x$ then $y = \log_b x$. *(§4.4.1)*

Log matching property — If two logs contain an entry with the same index and term, then (1) the entries store the same command, and (2) the logs are identical in all preceding entries. *(§26.7.4)*

Log-structured merge tree (LSM tree) — A storage structure that buffers writes in a sorted in-memory table (the **memtable**), periodically writes it out as an immutable sorted file (a **sorted string table, SSTable**), and merges SSTables in the background (**compaction**) into progressively larger sorted runs organized in **levels**. Introduced by O'Neil, Cheng, Gawlick and O'Neil [O'Neil et al. 1996]; the memtable/SSTable design popularized by Google's Bigtable [Chang et al. 2006] and implemented in LevelDB, RocksDB, Cassandra, HBase, ScyllaDB and many others. *(§13.8.1)*

Log-structured storage — A storage design in which every write — insert, update or delete — is appended to the end of a file, and existing data is never modified in place. Old versions become garbage that a background **compaction** (or merge) process removes by copying live data into new files. *(§13.7.1)*

Long polling — A technique in which the client sends an HTTP request that the server deliberately does not answer until either new data is available or a server-side timeout T_{hold} expires; on receiving any response, the client immediately sends the next request. *(§9.4.1)*

Low-level design (LLD) — A description of the internals of one component: its modules, classes or functions, data structures, algorithms, database schemas, API request and response formats, error handling and state transitions — at a level of detail from which an engineer can write code. *(§1.5.1)*

M

Maglev hashing — A consistent-hashing scheme that builds a **lookup table** of prime size M in which every entry names a backend. Each backend has its own pseudo-random **preference permutation** of table positions; backends take turns claiming their next preferred empty position until the table is full. A packet's backend is `table[hash(5-tuple) mod M]`: one array read [Eisenbud et al. 2016]. *(§19.4.8)*

MapReduce — A programming model and execution system in which the user writes two functions: **map**, which turns one input record into zero or more intermediate key–value pairs, and **reduce**, which receives one intermediate key with *all* values emitted for it and produces output. The system handles splitting, scheduling, regrouping by key, and failures [Dean and Ghemawat 2004]. *(§29.4.1)*

Matrix — A rectangular grid of numbers with m rows and n columns (an $m \times n$ matrix). A set of n embeddings of dimension d is naturally stored as an $n \times d$ matrix, one embedding per row. *(§4.9.3)*

Mean and median — The **mean** (arithmetic average) of n values is their sum divided by n. The **median** is the middle value after sorting (the average of the two middle values when n is even); half of the values are at or below it. *(§4.7.1)*

Memory hierarchy — The layered arrangement of storage technologies in a computer, from small, fast and expensive per byte (registers, caches) to large, slow and cheap per byte (disks, tape), in which each level acts as a cache for the level below it. *(§5.5.2)*

Merkle tree (hash tree) — A tree in which every leaf holds the hash of a data block and every interior node holds the hash of its children's hashes. The **root hash** summarizes the entire data set: changing any block changes the root [Merkle 1987]. *(§17.8.1)*

Message authentication code (MAC) — A short tag computed from a message and a secret key, $t = \mathrm{MAC}_K(m)$, such that without K it is infeasible to produce a valid tag for any new message. **HMAC** is the standard construction from a hash function: $\mathrm{HMAC}_K(m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$ [Krawczyk et al. 1997]. *(§10.4.6)*

Metadata service — The component that maps names to data: directory entries, file or object attributes (size, owner, permissions, version, checksum) and the locations of each chunk or fragment. It is on the path of every open, create, list and delete, while the data path bypasses it. *(§15.7.1)*

Mobile push notification — A message sent by an application's server to a platform-operated push service — Apple Push Notification service (**APNs**) for Apple platforms, Firebase Cloud Messaging (**FCM**) for Android (and optionally iOS and web) — which delivers it, on a best-effort basis, over a single persistent connection that the operating system maintains to that service on behalf of all apps. *(§9.8.1)*

Monotonic reads — A client never sees data move backwards: after reading some version, its later reads return that version or newer. *(§14.4.3)*

Multi-core processor — A single chip containing several complete cores, each with its own pipeline and private caches, sharing the last-level cache and memory. *(§5.3.6)*

Multipart upload — A protocol for uploading one large object as independently uploaded **parts**: the client *initiates* an upload and receives an upload ID, uploads numbered parts (in any order, in parallel, retrying any part), then *completes* the upload with the list of part numbers and their ETags, at which point the store assembles the object atomically. Until completion, or after an *abort*, the object is not visible. *(§15.8.1)*

Multi-primary replication — More than one node accepts writes for the same data and replicates to the others (also **multi-leader** or **active–active**). Concurrent uncoordinated

changes to the same row create **write conflicts** that must be detected and resolved. *(§14.5.1)*

Mutex (mutual-exclusion lock) — An object with two operations, `lock` and `unlock`, guaranteeing that at most one thread holds it at a time. A thread that calls `lock` while another holds it waits. *(§6.10.1)*

MVCC — A concurrency-control technique in which a write creates a new **version** of a row instead of overwriting it in place, and each transaction reads the versions visible in its **snapshot**: the set of transactions that had committed when the snapshot was taken. Readers therefore never block writers and writers never block readers; only writers of the same row wait for each other. *(§11.14.1)*

N

Narrow and wide dependencies — A **narrow dependency** exists when each partition of the parent RDD is used by at most one partition of the child (`map`, `filter`, `union`, a join of co-partitioned inputs). A **wide (shuffle) dependency** exists when a child partition may depend on many or all parent partitions (`groupByKey`, `reduceByKey`, a join of unpartitioned inputs), which requires a shuffle. *(§29.5.2)*

Natural and surrogate keys — A **natural key** is a candidate key with business meaning (email address, ISBN, country code). A **surrogate key** is a meaningless value generated by the system (an auto-increment integer, a UUID) whose only job is identification. *(§11.4.2)*

Near-real-time (NRT) search — A search system in which newly written documents become searchable after a short, bounded delay (typically one second to a minute) rather than immediately, because making them visible requires creating a new segment and reopening the searcher. *(§16.10.1)*

Network Address Translation (NAT) — A function, usually in a router or gateway, that rewrites the source and/or destination IP addresses (and usually ports) of packets as they pass, keeping a **translation table** so that replies can be rewritten back. The most common form, **source NAT with port translation** (NAPT, often just “NAT”), lets many hosts with private addresses share one public address [RFC 3022]. *(§7.13)*

Network partition — A network fault after which some nodes that are running correctly cannot exchange messages with some other running nodes for a period of time. (This book reserves “partition” alone for a subset of data, §14.6; a network fault is always a **network partition**.) *(§24.7.1)*

NewSQL (distributed SQL) — Relational databases that keep SQL and ACID transactions with serializable or strong isolation while scaling horizontally across nodes and re-

gions, by partitioning data into ranges replicated with a consensus protocol. The term was coined by analyst Matthew Aslett in 2011 [Pavlo & Aslett 2016]. *(§12.11)*

Non-functional requirement (NFR) — A requirement on *how well* the system performs its functions, or on constraints under which it operates, expressed as a measurable target — for example "p99 feed latency ≤ 300 ms at 2,000 RPS" or "monthly availability ≥ 99.9%". NFRs are also called **quality attributes** or, informally, the **"-ilities"** (scalability, availability, durability, …). *(§1.7.1)*

Normal forms (1NF to BCNF) — **1NF:** every column holds a single atomic value (no lists or repeating groups). **2NF:** 1NF, and no non-key column depends on only *part* of a composite candidate key. **3NF:** 2NF, and no non-key column depends on another non-key column (no transitive dependency); formally, for every non-trivial $X \to A$, either X is a superkey or A is part of some candidate key. **BCNF (Boyce–Codd normal form):** for every non-trivial $X \to A$, X is a superkey. BCNF removes 3NF's exception for key columns. *(§11.5.2)*

NoSQL database — A database that does not use the relational table-and-join model as its primary data model and typically trades some combination of cross-record transactions, ad-hoc querying and fixed schemas for horizontal scale-out, flexible schemas or availability during failures. The name is usually read as "not only SQL"; it describes a movement, not a specification. *(§12.3.1)*

NUMA (non-uniform memory access) — A memory architecture in which each processor socket (or each chiplet group within a socket) has its own directly attached memory. A core can access all memory in the machine, but accessing its **local** memory is faster than accessing **remote** memory attached to another socket. *(§5.10)*

O

OAuth 2.0 — An authorization framework that lets a third-party application obtain *limited* access to an HTTP service on behalf of a resource owner, by obtaining an **access token** from an authorization server instead of using the owner's credentials [RFC 6749]. *(§8.14.1)*

Offload ratio — The fraction of traffic a CDN serves without contacting the origin. **Request offload** counts requests; **byte offload** counts bytes. Both are a cache hit ratio (§20.3) measured at the boundary between the CDN and the origin. *(§21.5.1)*

Open and closed workload models — In an **open** model, new requests arrive at a rate that is independent of how fast the system answers: users on the internet do not wait for each other. In a **closed** model, a fixed population of N users (or load-generator threads) each sends a request, waits for the response, optionally pauses for a **think time** Z, and

only then sends the next. A **partly open** model has sessions that arrive openly but issue several requests each, closed-loop, within the session [Schroeder et al. 2006]. *(§23.5.2)*

OpenAPI Specification (OAS) — A vendor-neutral, machine-readable format (YAML or JSON) for describing HTTP APIs: paths, operations, parameters, request and response bodies (as JSON Schema), status codes and security schemes [OpenAPI Specification]. It grew out of Swagger and is maintained by the OpenAPI Initiative; version 3.1 aligned its schemas with JSON Schema, and version 3.2.0 was released in September 2025. *(§8.16)*

Operating system (OS) — The software layer that manages a computer's hardware (CPU, memory, storage, network) and provides running programs with safe, shared, abstract access to it. The privileged core of the OS that runs with full hardware access is the **kernel**. *(§6.3.1)*

Operation, history — An **operation** is one call by a client on a shared object, such as `write(x, 1)` or `read(x)`. It has an **invocation** event (the client sends the request, at real time t_{inv}) and, if the client hears back, a **response** event (at t_{resp}, carrying the return value). A **history** is the sequence of invocation and response events observed in one execution. An operation without a response is **pending**: it may or may not have taken effect. *(§25.3.1)*

Order of magnitude — A factor of ten. Two quantities differ by k orders of magnitude when their ratio is about 10^k. Reasoning in orders of magnitude means estimating only the *power of ten* of a quantity (is it 10, 1,000 or 1,000,000 requests per second?) because that, not the exact value, decides the architecture. *(§2.10.1)*

Origin shielding — Routing every cache miss for an origin through one designated cache tier (the **shield**), so that the origin sees at most one request per object per freshness period, regardless of how many edge PoPs missed. *(§21.5.3)*

Out-of-order (OoO) execution — A technique in which the core executes instructions as soon as their inputs are ready rather than strictly in program order, while still making the results *appear* in program order (it **retires** them in order). *(§5.3.5)*

OWASP Top 10 — A consensus ranking of the most critical web application security risk categories, published by the Open Worldwide Application Security Project from vulnerability data contributed by testing organizations plus a community survey. This section uses the **2021 edition** [OWASP 2021]. A 2025 edition has since been published; it keeps broken access control at #1, folds SSRF into it, and adds categories for software supply-chain failures and mishandling of exceptional conditions, so the architectural controls below remain applicable. *(§10.7)*

P

PACELC — If there is a **P**artition, how does the system trade off **A**vailability and **C**onsistency; **E**lse, when the system is running normally, how does it trade off **L**atency and **C**onsistency? A system is labeled by its two choices, for example PA/EL or PC/EC. *(§25.9.1)*

Page — The fixed-size unit (typically 4–16 KiB) in which a storage engine reads, writes, caches and logs data. A page on disk is sometimes called a **block**; the engine keeps a copy of a page in memory in a **frame** of its buffer pool. *(§13.3.1)*

Page cache — The kernel's in-RAM cache of file contents, organized in pages. Reads are served from it when possible; writes normally modify pages in it (marking them **dirty**) and return immediately, with the kernel writing them back to storage later. *(§6.6.6)*

Partial failure — A state in which some components of a system have failed while others continue to work, so that an operation may be applied in some places and not others, and the working components cannot reliably tell which components failed. *(§24.8.1)*

Partition key, sort key and item collection — In DynamoDB every **item** (a record of up to 400 KB, as of 2026) has a primary key made of a **partition key** and, optionally, a **sort key**. DynamoDB hashes the partition key to choose the storage partition. All items sharing a partition key form an **item collection**, stored together and ordered by sort key, so a single **Query** can return a contiguous, sorted range of them. *(§12.4.3)*

Partition (shard) — A subset of a dataset's rows, stored and served together, such that every row belongs to exactly one partition. **Partitioning** spreads partitions over nodes; **sharding** is a synonym used mostly for separate database servers. This book uses *partition* and *shard* interchangeably and reserves **network partition** for network splits (§24.7). The columns that choose a row's partition form the **partition key** (or **shard key**). *(§14.6.1)*

Partition skew and salting — **Skew** is an uneven distribution of records across partitions, usually caused by a few very frequent keys (a celebrity user, a null or default value, a bot). **Salting** appends a random suffix from a small range to hot keys so their records spread over several partitions, then removes the suffix in a second, much smaller aggregation (or, for joins, replicates the matching rows of the other side once per salt value). *(§29.6.3)*

Percentile — The q-th percentile of a set of measurements is the value at or below which q% of the measurements fall. Latency percentiles are written **p50** (the median), **p95**, **p99** and **p99.9** (also written p999): the 50th, 95th, 99th and 99.9th percentiles. *(§4.8.1)*

Percentile latency (p50, p95, p99, p99.9) — The latency value below which the given percentage of requests fall. If p99 latency is 300 ms, 99% of requests completed in 300 ms or less and 1% took longer. p50 is the median. *(§1.7.1)*

Percolator transaction — A snapshot-isolated, multi-row, multi-table transaction built entirely by a client library on top of a store that offers only single-row atomic updates and multiple timestamped versions per cell. Ordering comes from a central **timestamp oracle**; atomicity comes from making one row's lock, the **primary lock**, the commit point for the whole transaction [Peng and Dabek 2010]. *(§28.7.1)*

Performance engineering — The practice of measuring, explaining and improving the **latency**, **throughput** and **resource efficiency** of a system against explicit targets, using repeatable methods rather than intuition. Its outputs are diagnoses (why is it slow?), changes (make it faster or cheaper) and predictions (how much capacity will we need?). *(§23.3.1)*

PKCE (Proof Key for Code Exchange, "pixy") — An extension in which the client creates a random secret, the **code verifier**, sends only its SHA-256 hash, the **code challenge**, with the authorization request, and must present the verifier when redeeming the authorization code. An attacker who intercepts the code cannot redeem it without the verifier [RFC 7636]. *(§8.14.3)*

Point of presence (PoP) — A CDN site in one metropolitan area or interconnection facility, containing load balancers, caching proxy servers with RAM and SSD storage, and connections to local networks. Large CDNs operate hundreds of PoPs; some also place servers inside Internet service provider (ISP) networks. *(§21.3.2)*

Polyglot persistence — Using several different data-storage technologies within one system, each chosen for the access pattern it serves best, rather than forcing all data into one database [Fowler 2011; Sadalage & Fowler 2012]. *(§12.13.1)*

Power of two choices (P2C) — For each request, pick **two** backends uniformly at random and send the request to the less loaded of the two (by in-flight count, EWMA latency or another load signal). *(§19.4.4)*

Presumed abort (PA) — A 2PC variant in which a coordinator that has no information about a transaction answers inquiries with "abort". Aborts therefore need no forced coordinator record, no forced participant record and no acknowledgments. *(§28.4.5)*

Primary–backup replication — One replica, the **primary**, receives all requests, executes them and propagates the result to one or more **backups**. If the primary fails, a backup is **promoted** to primary. It is also called **passive replication** [Alsberg and Day 1976; Budhiraja et al. 1993]. *(§27.4.1)*

Primary key in CQL — In Cassandra Query Language, `PRIMARY KEY ((a, b), c, d)` declares `(a, b)` as the **partition key** — hashed to choose the replicas and defining what is stored together — and `c, d` as **clustering columns**, which order rows *within* the partition on disk. *(§12.6.2)*

Priority inversion — A situation in which a high-priority thread is blocked waiting for a lock held by a low-priority thread, while a medium-priority thread, which needs no lock, preempts the low-priority one, so the high-priority thread effectively runs at the lowest priority. *(§6.11.5)*

Probability — A number between 0 and 1 that measures how likely an event is: 0 means impossible, 1 means certain. If an experiment is repeated many times, the probability of event A, written $P(A)$, is the long-run fraction of repetitions in which A happens. *(§4.6.1)*

Process — A running instance of a program: its code, its private **virtual address space**, one or more threads of execution, and the OS resources it holds (open files, sockets, signal handlers, credentials). *(§6.3.2)*

Process control block (PCB) — The kernel data structure that records everything needed to stop a process and later resume it exactly where it left off. In Linux it is `struct task_struct`. *(§6.3.3)*

Process state — The scheduling status of a process (or thread): whether it is running on a CPU, ready and waiting for a CPU, or blocked waiting for an event. *(§6.3.4)*

Profiler — A tool that attributes a resource (CPU time, wall-clock time, allocated bytes, lock wait time) to the code locations that consumed it. A **sampling profiler** interrupts the program at a fixed frequency and records the current call stack; a function's share of samples estimates its share of time. An **instrumenting** (deterministic, tracing) profiler inserts hooks at every function entry and exit and records exact call counts and durations. *(§23.4.1)*

Protocol — A set of rules two parties agree on for exchanging messages: message formats, the meaning of each field, and the actions each side takes on sending, receiving or timing out. A **protocol stack** is a set of protocols layered so that each uses the service provided by the layer below and offers a service to the layer above. *(§7.3.1)*

Proxy — An intermediary that terminates a client's connection and opens its own connection onward, relaying requests and responses. A **forward proxy** acts on behalf of **clients** and is chosen by them (or their network administrator). A **reverse proxy** acts on behalf of **servers** and is chosen by the site operator; clients usually do not know it exists. *(§7.12)*

Public key infrastructure (PKI) — The set of roles, policies and systems that bind public keys to identities. A **certificate** (X.509) is a document stating "this public key belongs to this name (for example `api.example.com`), valid from date A to date B", signed by a **certificate authority (CA)**. A client trusts a certificate if it can build a **chain** of valid signatures from it to a **root CA** certificate in its local **trust store**. *(§10.5)*

Q

Queueing system — A model in which **jobs** (requests, messages, packets) arrive at a **service center**, wait in a **queue** if all **servers** are busy, receive service, and depart. The **arrival rate** λ is the average number of jobs arriving per second; the **service rate** μ is the average number of jobs one server can complete per second when it is never idle, so the average **service time** is $S = 1/\mu$. *(§4.10.1)*

Quorum — A set of nodes whose agreement is sufficient for an operation to take effect. A **majority quorum** in a cluster of N voting nodes is any set of more than $N/2$ nodes. The essential **intersection property**: any two majority quorums share at least one node. *(§26.4.1)*

R

Race condition — A bug in which the result of a program depends on the relative timing or interleaving of concurrent operations on shared state. A **data race** is the specific case where two threads access the same memory location concurrently, at least one access is a write, and nothing orders them. *(§6.9)*

Raft term — A logical clock that divides time into consecutive, numbered periods. Each term begins with an election; if a candidate wins, it is the **leader** for the rest of the term, and there is at most one leader per term. Every RPC carries the sender's term; a node that sees a higher term immediately adopts it and becomes a follower; a request with a lower term is rejected. *(§26.7.1)*

Rate limiting — Enforcing an upper bound on how many requests (or how much work) a given **client identity** may submit to a resource over time. Requests beyond the bound are rejected (or, less often, delayed). The bound is usually stated as a **rate** (for example 100 requests per minute) plus a **burst** allowance. *(§22.3.1)*

Read repair and anti-entropy — **Read repair** fixes stale replicas discovered while serving a read: the coordinator writes the newest version back to replicas that returned older ones. **Anti-entropy** is a background process that compares whole replicas, typically with Merkle trees, and copies the differences [DeCandia et al. 2007; Demers et al. 1987]. *(§27.13.1)*

Read-through — The application asks only the cache; on a miss the *cache* loads the value from the database through a configured loader, stores it and returns it. *(§20.7.3)*

Read-your-writes consistency — After a client writes a value, its own later reads return that value or a newer one. Other clients get no such promise. *(§14.4.3)*

Rebalancing — Moving partitions, or parts of them, between nodes so that data and load stay even as nodes join, leave or fail and as data grows, while staying available and moving no more data than necessary. *(§14.9.1)*

RED method — For every **service**, monitor **R**ate (requests per second), **E**rrors (failed requests per second) and **D**uration (the distribution of request latency, as a histogram from which percentiles are read). Tom Wilkie proposed it while at Weaveworks as a service-level complement to USE [Wilkie 2018]. *(§23.3.3)*

Reduce-side (sort-merge) join and broadcast (map-side) join — In a **reduce-side join**, both inputs are mapped to `(join key, tagged record)`, shuffled so that all records with the same key meet at one reducer, sorted, and combined there. In a **broadcast join**, the smaller input is copied in full to every map task, loaded into an in-memory hash table, and each record of the larger input is joined locally by lookup, with no shuffle of the larger input. *(§29.4.4)*

Register — A tiny, extremely fast storage slot inside a core, typically 64 bits wide for general-purpose registers. x86-64 exposes 16 general-purpose integer registers to programs; ARMv8-A (AArch64) exposes 31. Reading a register costs essentially nothing beyond the instruction that uses it. *(§5.3.2)*

Relation — A **relation** is a named set of **tuples** that share the same **attributes**. Each attribute has a name and a **domain** (the set of allowed values, such as 64-bit integers). In SQL a relation is a **table**, a tuple is a **row** and an attribute is a **column**. The relation's name and attributes together form its **schema**; the set of rows at a moment is its **instance**. *(§11.3.1)*

Relevance score — A number computed for each (query, document) pair that estimates how well the document satisfies the query. A **ranking function** computes it; the engine returns the k highest-scoring documents. *(§16.5.1)*

Reliability, MTBF, MTTR and failure rate — **Reliability** $R(t)$ is the probability that a component operates without failure for a period of length t. The **mean time between failures (MTBF)** is the average operating time from the end of one repair to the next failure (for non-repairable parts the same quantity is called **MTTF**, mean time to failure). The **mean time to repair (MTTR)** is the average time from failure to restored service, including detection, diagnosis and repair. The **failure rate** λ is the average number of failures per unit of operating time. *(§4.15.1)*

Remote procedure call (RPC) — A mechanism that lets a program call a function in another process, usually on another machine, with syntax that resembles a local function call. Generated **stubs** serialize the arguments, send them over the network, and deserialize the result [Birrell and Nelson 1984]. *(§8.8.1)*

Replica recovery — Returning a replica to full membership after a crash or replacement, by **log catch-up** (replaying missed log entries), **snapshot plus log** (installing a copy of the state, then replaying from the snapshot's position) or **rebuild** (copying all data from peers). *(§27.12.1)*

Replicated state machine (RSM) — A service implemented as a deterministic state machine that is copied onto several nodes. Each node applies the same sequence of commands, taken from a replicated **log**, in the same order, so every node passes through the same sequence of states and produces the same outputs [Schneider 1990]. *(§26.3.2)*

Replication — Keeping a copy of the same data on more than one node. Each copy is a **replica**. In **primary–replica replication**, exactly one node, the **primary** (called the "master" in older documentation), accepts writes and streams its changes to the **replicas**, which apply them in the same order and serve reads. *(§14.3.1)*

Replication lag — The delay between a change committing on the primary and becoming visible on a replica, measured in seconds, bytes of log or transactions. *(§14.4.2)*

Replication log formats — What the log describes: the SQL statements executed (**statement-based**), the changed rows (**row-based**), the storage engine's page-level byte changes (**physical**, or WAL shipping), or decoded, engine-independent row changes per table (**logical**). *(§14.4.5)*

Replication modes — In **synchronous replication**, the primary reports a commit only after at least one designated replica has durably received (or applied) it. In **asynchronous replication**, it reports success after its local commit and ships the change afterwards. **Semi-synchronous replication**, MySQL's middle ground, waits until a replica has *received and logged* the change and falls back to asynchronous mode after a timeout. *(§14.4.1)*

Replication strategy — The rule that decides *which* replicas may accept a write, *in what order* the replicas apply writes, and *when* a write counts as done. The five families are **primary–backup**, **chain replication**, **multi-leader**, **leaderless** and **consensus-based** replication. *(§27.3.1)*

Replication topology — The graph along which leaders forward writes to each other: a **circular** ring, a **star** with a hub, or **all-to-all**. *(§27.6.1)*

Request–response lifecycle — The complete sequence of network and processing steps between a client issuing a request and receiving the full response, including name resolution, connection establishment, routing through intermediaries, application processing, data access and the return path. *(§8.5.1)*

Requirement — A statement of what the system must do (a **functional requirement**, e.g. "a customer can cancel an order before the restaurant accepts it") or how well it

must do it (a **non-functional requirement**, e.g. “99% of order placements complete within 800 ms”). A requirement is **testable** if you could write an automated check or a measurement that proves it is met. *(§2.5.1)*

Reservoir sampling — An algorithm that maintains a uniform random sample of k items from a stream whose length n is not known in advance, using $O(k)$ memory and a single pass: after any number of items, every item seen so far is in the sample with probability k/n [Vitter 1985]. *(§17.12.1)*

Resilient Distributed Dataset (RDD) — A read-only, partitioned collection of records that can be created only by deterministic operations (**transformations** such as `map`, `filter`, `join`) on stable storage or on other RDDs. Each RDD records its **lineage**, the graph of transformations that produced it, so a lost partition can be recomputed from its parents instead of being restored from a replica [Zaharia et al. 2012]. *(§29.5.1)*

REST (Representational State Transfer) — An architectural style for distributed hypermedia systems, defined by Roy Fielding in his 2000 doctoral dissertation as a set of constraints on how components interact [Fielding 2000]. In everyday usage, a “REST API” is an HTTP API that models its domain as **resources** identified by URIs and manipulated with the standard HTTP methods. *(§8.6.1)*

Round robin (RR) — Send the k-th request to backend $k \bmod N$. **Weighted round robin (WRR)** gives backend i an integer weight w_i and sends it a share $w_i / \sum_j w_j$ of requests in each cycle. *(§19.4.1)*

Routing and forwarding — **Forwarding** is the per-packet act of choosing an outgoing interface for a packet based on its destination address and the **forwarding table**. **Routing** is the control-plane process of building those tables by exchanging reachability information with other routers. *(§7.5.1)*

RUM conjecture — For any access method, bounding two of **R**ead overhead, **U**pdate overhead and **M**emory (space) overhead forces the third to be worse: you can optimize for at most two [Athanassoulis et al. 2016]. *(§13.11.1)*

S

Safe method — A method whose semantics are read-only: the client does not request, and does not expect, any state change on the server [RFC 9110 §9.2.1]. Logging or metrics side effects are allowed. *(§8.6.3)*

Saga — A long-lived transaction written as a sequence of local transactions $T_1, T_2, \ldots, T_n$, each committed independently, where each T_i has a **compensating transaction** C_i that semantically undoes it. Either $T_1 \ldots T_n$ all complete, or $T_1 \ldots T_j, C_j \ldots C_1$ run [Garcia-Molina and Salem 1987]. *(§28.6)*

Scalability — The ability of a system to handle a growing workload by adding resources, with throughput rising roughly in proportion to the resources added and latency staying within its objective. A system is *not* scalable for a workload if adding resources yields little or no extra throughput, or if the cost per unit of work rises sharply. *(§18.3.1)*

Scale cube — A model by Martin Abbott and Michael Fisher that classifies every scaling technique along three independent axes [Abbott and Fisher 2015]: the **X axis**, cloning identical copies of a service or its data behind a load balancer; the **Y axis**, splitting by function, service or resource so that each part handles different work; and the **Z axis**, splitting by customer, requester or other key so that each part handles the same work for a subset of the data or users. *(§18.10.1)*

Scaling bottleneck — The resource that limits throughput as load or node count grows: the resource with the largest service demand per unit of capacity (§2.6). For a horizontally scaled system, it is usually a resource that *all* nodes share and that cannot itself be scaled out by adding application nodes. *(§18.7.1)*

Search engine (as a data store) — A system such as Elasticsearch, OpenSearch or Apache Solr that stores JSON documents and builds an **inverted index** — a map from each term to the list of documents containing it — so it can answer full-text, relevance-ranked, faceted and fuzzy queries. *(§12.9)*

Secret — Any value that grants access and must be kept confidential: database passwords, API keys, OAuth client secrets, TLS and signing private keys, encryption keys, webhook HMAC keys. **Secrets management** is the practice of storing, distributing, rotating, auditing and revoking secrets so that they are available to the workloads that need them and to nobody else. *(§10.8)*

Security property — A statement about what must remain true about a system even when an adversary is actively trying to violate it. The classic triad is **confidentiality** (only authorized parties can read data), **integrity** (data and behavior cannot be changed without detection) and **availability** (authorized parties can use the system when they need it), often abbreviated **CIA**. System designers add **authentication** (knowing who is acting), **authorization** (deciding what that actor may do) and **non-repudiation** (an actor cannot credibly deny an action they took). *(§10.3.1)*

Segment — A self-contained, immutable mini-index (its own dictionary, postings, stored fields, norms and doc values) covering a subset of documents. A Lucene index is a set of segments plus a **commit point** file (`segments_N`) listing which segments are live; a search runs over every live segment and combines the results. *(§16.6.1)*

Semaphore — An integer counter with two atomic operations introduced by Dijkstra: **wait** (P, or *acquire*) decrements it and blocks if the result would be negative; **signal** (V, or *release*) increments it and wakes a waiter [Dijkstra 1965]. *(§6.10.2)*

Sequential consistency — A history is **sequentially consistent** if there is a single total order of all operations that is a legal sequential history of the objects and that respects each client's **program order** (the order in which that client issued its own operations). In Lamport's words, "the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program" [Lamport 1979]. *(§25.4.1)*

Serialization (encoding) — Translating an in-memory data structure into a sequence of bytes for storage or transmission; **deserialization (decoding)** is the reverse [Kleppmann 2017, ch. 4]. *(§8.9.1)*

Server-Sent Events (SSE) — A W3C/WHATWG-standardized mechanism in which a client opens one long-lived HTTP request and the server streams a sequence of UTF-8 text events down the response body using the `text/event-stream` format; browsers expose it through the `EventSource` API, which parses events and reconnects automatically, resuming from the last received event ID [WHATWG HTML §9.2]. *(§9.5.1)*

Server-side rendering (SSR) — The server produces complete HTML for each request; the browser displays it immediately and then loads JavaScript to make it interactive (**hydration**). *(§8.4.2)*

Session — Server-recognized state that links a series of requests to one authenticated user (or anonymous visitor) over time, typically identified by a cookie. *(§8.10.2)*

Session affinity (sticky sessions) — A balancer policy that sends all requests of one client session to the same backend, identified by a cookie, the client's IP address or a request key. *(§19.6)*

Session guarantees — **Read-your-writes (RYW):** if a session writes w and later reads, the read reflects w (or a later write). **Monotonic reads (MR):** if a session reads a value reflecting a set of writes, later reads reflect at least that set; the session never sees time go backwards. **Monotonic writes (MW):** a session's writes are applied at every replica in the order the session issued them. **Writes-follow-reads (WFR):** if a session reads a value produced by write w_1 and then writes w_2, every replica applies w_1 before w_2 (the new write is ordered after what the session saw). *(§25.6.1)*

Set-associative cache — A cache divided into S sets of W slots (**ways**). A line at a given address may be stored in any of the W ways of exactly one set. $W = 1$ is a **direct-mapped** cache; a cache with a single set holding every line is **fully associative**. *(§5.4.4)*

Short polling — A technique in which the client sends an ordinary HTTP request at a fixed interval T to ask whether new data exists; the server answers immediately, either with the new data or with an empty response, and the connection's request ends. *(§9.3.1)*

Shuffle — The all-to-all redistribution of intermediate data between two stages, in which every task of the producing stage writes its output partitioned by key and every task of the consuming stage reads its partition from every producer. It costs serialization, local disk writes and reads, and network transfer, and it is a barrier between stages. *(§29.6.2)*

SIMD (single instruction, multiple data) — CPU instructions that apply the same operation to several data elements packed into one wide **vector register** at once — for example, adding eight pairs of 32-bit floats in a single instruction. *(§5.11.1)*

Simultaneous multithreading (SMT) — A technique in which one physical core runs two (occasionally more) instruction streams at once, interleaving them through the same execution units. Intel markets its implementation as **Hyper-Threading Technology**. Each stream appears to the operating system as a separate **logical CPU** (or **hardware thread**). *(§5.3.6)*

Single-decree (basic) Paxos — A consensus algorithm in which **proposers** suggest values, **acceptors** vote on them and **learners** find out the decided value. A value is **chosen** when a majority of acceptors has accepted it with the same proposal number. Paxos guarantees safety under any message delays, loss or reordering, and makes progress whenever one proposer can run both phases without interference from another. *(§26.6.1)*

Single-page application (SPA) / client-side rendering (CSR) — The server sends a nearly empty HTML shell plus a JavaScript bundle; the JavaScript calls JSON APIs and builds the page in the browser. Navigation afterwards happens without full page loads. *(§8.4.2)*

Skip list — An ordered linked list augmented with extra levels of “express” pointers. Each node is promoted to the next level with probability p (typically 1/2 or 1/4), so level i contains about a fraction p^i of the nodes. Search, insert and delete take expected $O(\log n)$ time [Pugh 1990]. *(§17.7.1)*

Sliding window counter — A limiter that keeps fixed-window counters for the current and previous windows and estimates the count over the last W seconds by weighting the previous window’s count by the fraction of it that still overlaps the sliding interval. *(§22.7.2)*

Sliding window log — A limiter that stores the timestamp of every admitted request per key and admits a new request only if fewer than L timestamps fall in the interval $(t - W, t]$. *(§22.7.1)*

Sloppy quorum and hinted handoff — In a **sloppy quorum**, a write that cannot reach W of a key’s N home replicas is accepted by the next healthy nodes on the ring instead, and still counts toward W. Each stand-in stores the write with a **hint** naming the intended home replica and **hands it off** when that replica recovers [DeCandia et al. 2007]. *(§27.9.1)*

Slotted page — A page layout in which a **header** is followed by an array of **slots** (also called line pointers or item IDs) that grows from the front, while variable-length **records** (tuples) are packed from the back. Each slot stores the offset and length of one record; free space sits in the middle. *(§13.3.2)*

Software architecture — "The set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both" [Bass et al. 2021]. The international standard for architecture description defines it as the fundamental concepts or properties of a system in its environment, embodied in its elements, relationships, and in the principles of its design and evolution [ISO/IEC/IEEE 42010]. *(§1.4.1)*

Solid-state drive (SSD) — A storage device with no moving parts that stores data in **NAND flash** memory cells, managed by an on-board controller running a **flash translation layer** (FTL). *(§5.6.2)*

Spaced repetition — Reviewing material at increasing intervals, so that each review happens around the time you would otherwise begin to forget it. The underlying finding, that spacing study sessions apart beats massing them together, is the **spacing effect**. *(§3.4.3)*

Speedup — The ratio of the time a job takes on one processor (or node) to the time it takes on N: $S(N) = T(1)/T(N)$. Perfect, **linear** speedup is $S(N) = N$. *(§4.12.1)*

Split brain — A failure in which two or more parts of a cluster each believe they are the authoritative primary (or leader) and accept conflicting writes, so the system's state diverges. *(§26.12.1)*

SQL — The Structured Query Language, standardized by ANSI/ISO, with sublanguages for defining schema (DDL: `CREATE`, `ALTER`, `DROP`), manipulating data (DML: `SELECT`, `INSERT`, `UPDATE`, `DELETE`) and controlling transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). Every major database implements the core standard plus its own dialect. *(§11.6.1)*

Stateful service — A service whose instances hold data that must survive between requests and that clients or other instances depend on: a database, a cache node, a WebSocket server holding live connections, or an application server keeping sessions in memory. *(§18.5.1)*

Stateless service — A service that keeps no client-specific data in its own memory between requests. Every request carries, or lets the service look up from a shared store, everything needed to process it. *(§8.3.2)*

Static site generation (SSG) — HTML is produced at build time and served as static files, usually from a CDN (Chapter 21). Pages that change per user cannot be fully static. *(§8.4.2)*

Storage tier and lifecycle policy — A **storage tier** (storage class) is a price/performance point for the same data: **hot** (frequent access, highest storage price, no retrieval fee), **warm** or infrequent access (cheaper storage, per-GB retrieval fee, minimum storage duration), **cold** (cheaper again, slower or costlier retrieval) and **archive** (cheapest storage, retrieval in hours). A **lifecycle policy** is a set of rules, usually by prefix, tag or age, that automatically transitions objects between tiers and expires (deletes) them. *(§15.9)*

Stream processing, event time and processing time — **Stream processing** runs a long-lived computation over an **unbounded** dataset, one that grows forever, producing results incrementally. Each record has an **event time**, when the event actually happened (stamped by the device or service that produced it), and a **processing time**, when the system observes it. The difference is the record's **event-time skew**, which is variable and unbounded in the worst case. *(§29.8.1)*

STRIDE — A mnemonic for six threat categories, developed at Microsoft in 1999 by Loren Kohnfelder and Praerit Garg and popularized by Shostack [Shostack 2014]: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service and **E**levation of privilege. Each category is the violation of one security property. *(§10.3.4)*

Superscalar execution — A core that can issue and complete more than one instruction per cycle by having several execution units working in parallel. *(§5.3.5)*

Symmetric encryption — Encryption in which the same secret key is used to encrypt and decrypt. A **block cipher** such as AES (Advanced Encryption Standard) encrypts fixed 128-bit blocks under a 128-, 192- or 256-bit key; a **mode of operation** turns the block cipher into a scheme for messages of any length. *(§10.4.2)*

System — A set of interacting **components** (programs, machines, data stores, networks and people) that together deliver a capability no single component delivers alone. Components interact only through **interfaces**: agreed formats and protocols for exchanging requests, responses and data. *(§1.3.1)*

System boundary — The line that separates what you are designing and are responsible for from everything outside it: users, other teams' systems and third-party services. You control what is inside. You can only *depend on* what is outside. *(§2.4.1)*

System call (syscall) — A controlled entry from user mode into the kernel to request a privileged service, such as reading a file, sending on a socket, allocating memory or creating a process. Linux x86-64 has a few hundred system calls, each identified by a number [Linux man-pages: syscalls(2)]. *(§6.7)*

System design — The process of deciding the components of a software system, their responsibilities, their interfaces, how data flows and is stored between them, and how they are deployed — so that the system meets its functional requirements *and* its non-func-

tional requirements (performance, scalability, availability, durability, security, cost and others) under stated assumptions about load and failure. The output of the process is also called the *system design*: a set of decisions, diagrams and documents. *(§1.3.2)*

System design interview — A 45–60-minute interview in which a candidate designs a system from an open-ended prompt (for example, “design a photo-sharing service”) in conversation with an interviewer, who evaluates the candidate’s process, technical judgment and communication rather than a single correct answer. *(§1.11.1)*

System model — A precise statement of what a distributed algorithm may assume about (1) **timing**: bounds on message delay, processing speed and clock drift, and (2) **faults**: how nodes and links may misbehave. Proofs of correctness hold only within the stated model. *(§24.6.1)*

T

TCP — A connection-oriented transport protocol that provides a reliable, ordered, full-duplex **byte stream** between two endpoints identified by (IP address, port) pairs. It achieves reliability with sequence numbers, acknowledgments and retransmission; it protects a slow receiver with **flow control**; and it protects the network with **congestion control**. *(§7.7.1)*

Thread — An independent sequence of execution within a process, with its own program counter, registers and stack, sharing the process’s address space, heap, globals and open files with the process’s other threads. *(§6.3.5)*

Threat model — A structured description of a system, the **assets** in it that are worth protecting, the **adversaries** who might attack it and their capabilities, the **threats** (ways a security property could be violated), and the **mitigations** chosen for each threat [Shostack 2014]. *(§10.3.2)*

Three-phase commit (3PC) — A commit protocol that inserts a **pre-commit** phase between voting and committing, so that no participant can commit until every operational participant knows that all voted yes. Under a synchronous system model with reliable failure detection and no network partitions, it lets the surviving participants always reach a decision without the failed coordinator [Skeen 1981]. *(§28.5.1)*

Time-of-day clock and monotonic clock — A **time-of-day** (wall-clock) clock returns the current date and time, for example as seconds since the Unix epoch (Linux `CLOCK_REALTIME`, Python `time.time()`). It is synchronized with other machines and *can jump* forward or backward when corrected. A **monotonic** clock (Linux `CLOCK_MONOTONIC`, Python `time.monotonic()`) only moves forward at a steady rate from an arbitrary starting point. It is meaningless across machines but correct for measuring durations on one machine. *(§24.9.1)*

Timeout and deadline — A **timeout** is a relative duration after which a caller stops waiting for one operation (for example 300 ms). A **deadline** is an absolute point in time by which the *whole* request must complete; it is passed along the call chain so that every downstream call knows how much time remains. *(§22.11)*

Time-series database (TSDB) — A database optimized for sequences of timestamped measurements. Data is organized into **series**, each identified by a metric name plus a set of **labels** (tags) such as `{host="web-3", region="eu"}`, and each series is an append-only list of **samples** `(timestamp, value)`. *(§12.8.1)*

Time to live (TTL) — The maximum time an entry may be served after it was stored. After the TTL the entry is treated as absent (or, with stale-while-revalidate, as stale). Redis removes expired keys lazily when they are accessed and actively by periodically sampling keys that have a TTL [Redis Docs: EXPIRE]. *(§20.8)*

TinyLFU — An **admission policy**: when the cache is full and a new candidate arrives, admit it only if its estimated access frequency exceeds that of the victim the eviction policy would remove. Frequencies are estimated by a small approximate sketch (a Count-Min sketch with 4-bit counters, §17.6) over a sample of recent requests, and **aged** by halving every counter after a fixed number of requests [Einziger, Friedman & Manes 2017]. *(§20.9.3)*

Token bucket — A rate limiter that keeps a counter of **tokens**, refilled continuously at **rate** r up to a maximum **capacity** (burst size) b. Each request consumes one token (or a cost c); a request that finds too few tokens is rejected. The long-run admitted rate is at most r, and up to b requests may be admitted at once. *(§22.4.1)*

Torn write (torn page) — A page write that is interrupted by a crash so that part of the page on disk is new and part is old. It arises because the database page (8 or 16 KiB) is larger than the unit the device or file system writes atomically (commonly 4 KiB or 512 bytes). *(§13.12.2)*

Trade-off — A situation where improving one desired property of a system makes another worse. Examples: lower latency at higher cost, stronger consistency at lower availability during network partitions, faster delivery at higher operational risk. Every design decision is a trade-off. If you cannot name what an option makes worse, you have not understood it yet. *(§2.8.1)*

Transaction — A sequence of reads and writes that the database treats as one logical unit: either all of its effects become permanent together (**commit**) or none of them do (**abort** or **rollback**), and concurrently running transactions are isolated from each other's intermediate states. *(§11.11.1)*

Transport Layer Security (TLS) — A protocol that gives two communicating applications **confidentiality** (eavesdroppers cannot read the data), **integrity** (tampering is de-

tected) and **authentication** (at least the server proves its identity, usually with an X.509 certificate). **HTTPS** is HTTP carried over TLS, on TCP port 443 by default. TLS 1.3 is specified in RFC 8446 (2018). *(§7.10.1)*

Trie (prefix tree) — A tree in which each edge is labeled with a character and each node represents the string spelled by the path from the root; nodes that end a stored key are marked. Looking up a key or a prefix of length L takes $O(L)$ steps, independent of how many keys are stored [Fredkin 1960]. *(§17.11.1)*

TrueTime — Google's clock API that returns time as an interval with a guaranteed error bound instead of a single value. `TT.now()` returns $[earliest, latest]$, guaranteed to contain the absolute time at which it was called; `TT.after(t)` is true if t has definitely passed, and `TT.before(t)` is true if t has definitely not arrived. The half-width of the interval is the **uncertainty** ε [Corbett et al. 2012]. *(§28.8.1)*

Trust boundary — A line in a system where the level of trust changes: data or requests crossing it come from a party that the receiving side does not fully control or trust. Examples: the internet-to-load-balancer edge, the browser-to-server connection, a service calling a third-party API, a tenant's code running on shared infrastructure. *(§10.3.2)*

Two generals problem — Two armies camped on opposite hills must attack a city in the valley at the same time; either attacking alone is defeated. Their generals can communicate only by messengers who cross the valley and may be captured. The problem is to design a messaging protocol after which both generals are *certain* that they will attack together. No such protocol with a finite number of messages exists. *(§24.3.3)*

Two-phase commit (2PC) — An atomic commitment protocol in which one node, the **coordinator** (also called the transaction manager), first asks every participant (resource manager) to **prepare**, that is, to make its part of the transaction durable while keeping it invisible and to promise to commit on request; and then, only if every participant voted yes, durably records a **commit** decision and tells all participants to commit. Any no vote or timeout leads to abort [Gray 1978; Bernstein et al. 1987]. *(§28.4.1)*

Two-phase locking (2PL) — A protocol in which each transaction acquires locks (shared **S** for reading, exclusive **X** for writing) before accessing data and, once it releases any lock, acquires no new ones: a *growing* phase followed by a *shrinking* phase. **Strict 2PL** holds all X locks until commit or abort. Any schedule produced by 2PL is serializable [Eswaran et al. 1976]. *(§11.15.1)*

U

UDP — The User Datagram Protocol [RFC 768] is a connectionless transport protocol that adds only port numbers, a length and a checksum to IP. Its 8-byte header provides

process-to-process delivery of individual **datagrams** with no handshake, no acknowledgments, no retransmission, no ordering, and no flow or congestion control. *(§7.8)*

USE method — For every **resource**, check **U**tilization (the fraction of time the resource was busy, or the fraction of its capacity in use), **S**aturation (the amount of extra work queued because the resource could not service it) and **E**rrors (the count of error events) [Gregg USE]. A resource is any physical or software component with finite capacity: CPUs, memory, disks, network interfaces, controllers and interconnects, but also locks, thread pools, connection pools and file descriptors. *(§23.3.2)*

V

Varint — A variable-length encoding of an unsigned integer in base 128: each byte carries 7 bits of the number, least-significant group first, and the most significant bit (MSB) of each byte is a continuation flag — 1 means "another byte follows" [Protocol Buffers Docs: Encoding]. *(§8.9.2)*

Vector — An ordered list of d numbers, $\mathbf{a} = (a_1, a_2, \ldots, a_d)$, where d is the **dimension**. Geometrically, a vector is an arrow from the origin to a point in d-dimensional space; its **length** (or norm) is $\| \mathbf{a} \| = \sqrt{a_1^2 + \cdots + a_d^2}$. *(§4.9.1)*

Vector clock — With n processes, each process P_i keeps a vector $V_i[1..n]$, initially all zeros. Before each event, P_i increments $V_i[i]$; a send attaches a copy of V_i. On receiving a message carrying V_m, P_i sets $V_i[k] \leftarrow \max(V_i[k], V_m[k])$ for every k, then increments $V_i[i]$. *(§24.11.1)*

Vector database — A data store that indexes high-dimensional numeric vectors (**embeddings** produced by machine-learning models) and answers **approximate nearest neighbor (ANN) search** queries: "the k stored vectors most similar to this query vector", by cosine similarity, dot product or Euclidean distance. *(§12.10)*

Vertical scaling (scaling up) — Increasing the capacity of a single node by giving it more or faster CPU cores, memory, storage or network bandwidth, for example by moving a database from an 8-core to a 64-core machine. *(§18.3.1)*

Virtual memory — A mechanism in which every address a program uses is a **virtual address** that the hardware's **memory-management unit (MMU)** translates, using tables maintained by the kernel, into a **physical address** in RAM, or into a fault if no valid mapping exists. *(§6.6.1)*

Virtual node (vnode) — One of V ring positions belonging to the same physical node, typically obtained by hashing labels such as `db-3#vn17`; also called a **token**. *(§14.8.1)*

Volcano (iterator) model — An execution model in which a query plan is a tree of operators, each implementing the same interface — `open()`, `next()` and `close()` — and each `next()` call pulls one row from the operator's children on demand [Graefe 1994]. *(§11.9.2)*

W

Watermark — A watermark with value t is a statement flowing through the stream: "no more events with event time $\leq t$ are expected." It lets event-time operators decide that a window is complete. A **perfect** watermark is possible only with full knowledge of the sources; real systems use **heuristic** watermarks, such as the maximum event time seen minus a bound δ on out-of-orderness, which can be wrong. Events that arrive with event time behind the watermark are **late data**. *(§29.8.3)*

WebRTC — A set of IETF protocols (overview in RFC 8825 [Alvestrand 2021]) and a W3C browser API [W3C WebRTC] for real-time audio, video and arbitrary data directly between peers. Media travels over SRTP, data over SCTP data channels, both secured by DTLS, typically over UDP; peers discover a working network path using ICE with the help of STUN and TURN servers. *(§9.7.1)*

WebSocket — A protocol, standardized in RFC 6455 [Fette & Melnikov 2011], that starts as an HTTP/1.1 request, *upgrades* the underlying TCP connection, and thereafter carries a full-duplex sequence of framed messages in both directions with 2–14 bytes of framing overhead per frame. URLs use the schemes `ws://` (plain TCP, default port 80) and `wss://` (over TLS, default port 443). *(§9.6.1)*

Wide-column store — A database whose rows are identified by a key, sorted (globally or within a partition) by that key, and contain a sparse, variable set of columns grouped into **column families**; each row may have millions of columns and each cell may hold several timestamped versions. Bigtable, HBase, Cassandra and ScyllaDB belong to this family. *(§12.6.1)*

Write-ahead log (WAL) — An append-only file of **log records** describing every change to the database, written *before* the changed pages themselves reach disk. Each record is identified by a monotonically increasing **log sequence number (LSN)**, usually its byte offset in the log. Also called the redo log (InnoDB) or journal. *(§13.5.1)*

Write-around — Writes go directly to the database and bypass the cache; the cache is populated only by reads (usually cache-aside), and the written key is invalidated or left to expire. *(§20.7.6)*

Write-back — Writes update only the cache and are acknowledged immediately; the cache marks entries **dirty** and flushes them to the database asynchronously, in batches or after a delay. *(§20.7.5)*

Write-through — Every write goes to the cache, which synchronously writes it to the database before acknowledging; the cache is updated and the database is updated as one client-visible operation. *(§20.7.4)*

X

XA — The X/Open Distributed Transaction Processing standard interface between a **transaction manager** (TM, the coordinator) and **resource managers** (RMs: databases, message brokers), with an **application program** (AP) demarcating transactions. Its main calls are `xa_start` and `xa_end` (associate work with a global transaction ID, the XID), `xa_prepare`, `xa_commit`, `xa_rollback` and `xa_recover` (list in-doubt XIDs after a crash) [X/Open 1991]. *(§28.4.6)*

Z

Zero trust — A security model in which network location confers no trust: every request is authenticated, authorized and encrypted based on the identity of the user, the device and the workload, plus contextual signals, regardless of where it originates. NIST SP 800-207 describes it as moving defenses “from static, network-based perimeters to focus on users, assets, and resources” [NIST SP 800-207]. *(§10.10.1)*

Zipf (Zipf-like) distribution — A popularity law in which the item of popularity rank i is requested with probability proportional to $1/i^{s}$, where $s > 0$ is the **skew exponent**. Breslau et al. found that requests to web proxies follow a Zipf-like law with an exponent below 1 and that, as a consequence, hit ratio grows roughly logarithmically with cache size [Breslau et al. 1999]. *(§20.3.3)*

ZooKeeper — A replicated coordination service that exposes a small, file-system-like tree of data nodes (**znodes**) with linearizable writes, FIFO per-client ordering, ephemeral and sequential nodes, and change notifications (**watches**). Applications build locks, leader election, barriers and configuration management on top of these primitives [Hunt et al. 2010]. *(§26.8.1)*